

mybibli

Manuel utilisateur

Les auteurs de mybibli
<https://github.com/guycorbaz/mybibli>

Version 1.7.9 — 26 mai 2026

Distribué sous licence AGPL-3.0-or-later.

Ce manuel couvre mybibli version 1.3.1 et ultérieures (le plancher v1.1.0 reste obligatoire pour les nouvelles installations — voir le chapitre Installation pour le contexte du seed-gate).

Table des matières

Préface	1
1 Installation	3
1.1 Pré-requis système	3
1.2 Checklist de pré-installation	3
1.3 Méthode 1 — Base de données embarquée	4
1.4 Méthode 2 — Base de données externe (exemple NAS Synology)	6
1.5 Référence des variables d’environnement	7
1.6 Premier boot — l’assistant de mise en service	9
1.7 Vérification de l’installation	9
2 Configuration	10
2.1 Lieux de stockage et hiérarchie	10
2.2 Étiquettes : codes V et codes L	11
2.3 Matériel : douchette code-barres	12
2.4 Données de référence	13
2.5 Paramètres système	13
2.6 Utilisateurs	14
3 Usage au quotidien	15
3.1 La page d’accueil	15
3.2 Cataloguer un nouveau titre via code-barres	15
3.3 Rechercher dans la collection	16
3.4 Déplacer un volume vers un autre lieu	17
3.5 Prêts	17
3.6 Audit de la bibliothèque	18
3.7 Rangement après retours	18
3.8 Wish list	18
3.8.1 Ajouter une entrée	18
3.8.2 Parcourir en librairie	19
3.8.3 Marquer un livre comme acheté	19
3.8.4 Auto-détection au scan catalogue	19
4 Types de médias multiples	20
4.1 Livres	20
4.2 BD et comics	20
4.3 Sorties audio	21
4.4 Films et séries	21
4.5 Choisir le type de média au scan	21
5 Fournisseurs de métadonnées	22

5.1	La chaîne de fournisseurs	22
5.2	Clés d'API	23
5.3	Où vivent les clés	24
5.4	Santé des fournisseurs	24
5.5	Quotas et back-pressure	24
5.6	Dépannage : métadonnées vides	25
6	Rôles et permissions	26
6.1	Qui peut faire quoi	26
6.2	Sessions et timeout d'inactivité	27
6.3	Garde du dernier admin actif	27
6.4	Réinitialisation de mot de passe	27
6.5	Sessions anonymes	28
7	Sauvegarde et restauration	29
7.1	Quoi sauvegarder	29
7.2	Procédure de sauvegarde	29
7.3	Cadence recommandée	30
7.4	Procédure de restauration	31
7.5	Migration vers un nouvel hôte	31
8	Mise à jour et migration	33
8.1	Mises à jour de routine ($1.y \rightarrow 1.z$)	33
8.2	Sauter des versions intermédiaires	33
8.3	Le saut $1.0.x \rightarrow 1.1.0$	34
8.4	Lire les notes de version	35
8.5	Nouveautés de la 1.7.9	35
8.6	Nouveautés de la 1.7.8	37
8.7	Nouveautés de la 1.7.7	37
8.8	Nouveautés de la 1.7.6	38
8.9	Nouveautés de la 1.7.5	39
8.10	Rollback	40
9	Bonnes pratiques	41
9.1	Hiérarchie de stockage	41
9.2	Planches de codes V	42
9.3	Genre versus Dewey	42
9.4	Cadence d'audit	42
9.5	Comptes utilisateur	42
9.6	Prêts	43
9.7	Fournisseurs de métadonnées	43
9.8	Mises à jour	43
10	Dépannage	44
10.1	Lire les logs	44
10.2	L'application ne démarre pas	44
10.3	Impossible de se connecter	45
10.4	Utilisateurs résiduels dans la table <code>users</code> après l'assistant	45
10.5	Les métadonnées ne se résolvent jamais	46
10.6	La recherche renvoie trop peu de résultats	46
10.7	Images de couverture manquantes	47
10.8	Où signaler un bug	47

11 API & intégrations	48
11.1 Pourquoi une API HTTP ?	48
11.2 Créer une clé d'API	48
11.3 S'authentifier	49
11.4 Points d'entrée	49
11.4.1 GET /api/v1/titles	49
11.4.2 GET /api/v1/titles/{id}	50
11.4.3 GET /api/v1/genres, /locations, /series	50
11.4.4 GET /api/v1/dewey/{prefix}	50
11.4.5 PATCH /api/v1/titles/{id}	50
11.5 Exemples	51
11.5.1 curl	51
11.5.2 Python	51
11.5.3 Une boucle de classificateur LLM	51
11.6 Révoquer une clé	52
11.7 Ce que l'API n'expose <i>pas</i>	52
12 Opérations & débogage	53
12.1 Où sont les journaux	53
12.2 Lire les journaux depuis l'hôte	53
12.3 Niveaux de journalisation	54
12.4 Lire le JSON structuré	55
12.5 Joindre un extrait de log à un rapport de bug	55
12.6 Rétention et rotation	55
13 Glossaire	57
Index	60

Préface

À qui ce manuel s'adresse

Ce manuel s'adresse à la personne qui installe, configure et exploite une instance mybibli pour son foyer, une petite association ou une bibliothèque communautaire. Il suppose que vous savez :

- ouvrir un terminal et lancer des commandes `docker compose` ;
- modifier un fichier texte de configuration dans l'éditeur de votre choix ;
- lire une URL imprimée dans une ligne de log et l'ouvrir dans un navigateur.

Aucune connaissance de Rust, MariaDB ou HTMX n'est requise. Le manuel se limite délibérément au rôle d'*exploitant* — compiler mybibli depuis les sources ou modifier son code est couvert par CLAUDE.md et les artefacts de planification du dépôt.

Conventions

Commandes. Les fragments de shell sont présentés dans un encadré à largeur fixe :

```
docker compose up -d
```

Fichiers de configuration. Les extraits de fichiers utilisent le même style, le nom du fichier étant indiqué dans le texte environnant. Les commentaires à l'intérieur du fragment utilisent le marqueur de commentaire propre au fichier.

Encadrés. Trois types d'encadrés signalent les passages importants :

Note

Une *note* fournit un contexte ou un éclairage qui aide à comprendre pourquoi quelque chose fonctionne comme cela. La sauter ne pose pas de problème.

Astuce

Une *astuce* suggère une pratique qui rend l'usage quotidien plus fluide. Essayez-la une fois que les bases sont maîtrisées.

Attention

Un *avertissement* signale une étape où une erreur peut entraîner une perte de données, vous verrouiller hors du panneau d'administration ou exposer l'instance à des accès non autorisés. Lisez ces encadrés en entier.

Renvois croisés. Les chapitres et sections sont liés dans le PDF — cliquez sur un titre ou un numéro de page en bleu pour y sauter. L'[index](#) en fin d'ouvrage renvoie chaque terme important à sa page de premier usage.

Demander de l'aide

La source de vérité canonique pour mybibli est le dépôt GitHub :

<https://github.com/guycorbaz/mybibli>

Ouvrez une *issue* sur le dépôt `guycorbaz/mybibli` lorsque vous rencontrez un bug, une lacune dans la documentation ou une fonctionnalité manquante. Les modèles d'issue demandent la version (visible en pied de page du panneau d'administration), votre méthode d'installation et les étapes pour reproduire le problème.

Couverture du manuel

Ce manuel couvre mybibli **1.1.0 et versions ultérieures**. Les versions antérieures comportaient un bug de sécurité où l'assistant de première mise en service était silencieusement contourné — voir le chapitre [8](#) pour le chemin de mise à jour depuis 1.0.x.

Bon catalogage.

Chapitre 1

Installation

Ce chapitre vous guide pour installer mybibli sur votre propre matériel. La cible de déploiement de référence est un serveur domestique, un NAS, ou toute machine Linux toujours allumée capable de faire tourner Docker.

1.1 Pré-requis système

- **Système d'exploitation** : toute distribution Linux, macOS, ou hôte Windows capable de faire tourner Docker. Le noyau hôte doit prendre en charge cgroups v2 (toute distribution à partir de 2021).
- **Docker** : version 24 ou ultérieure, avec le plugin `compose`. Vérifiez avec `docker compose version`.
- **Disque** : au moins 1 Go libre pour l'image de l'application et la base de données. Les images de couverture ajoutent environ 50 Ko par titre.
- **Mémoire** : 512 Mo suffisent pour une bibliothèque de taille familiale (quelques milliers de titres) ; 1 Go offre une marge confortable pour le pipeline de récupération de métadonnées.
- **Réseau** : HTTPS sortant vers les fournisseurs de métadonnées que vous choisissez d'utiliser (voir chapitre 5). Les déploiements en LAN uniquement fonctionnent sans aucune redirection de port entrante.

Attention

N'installez pas mybibli en version inférieure à 1.1.0. Toute image publiée portant un tag inférieur à 1.1.0 contient une migration d'amorçage qui crée les identifiants `admin/admin` et `librarian/librarian` à *chaque* installation neuve, y compris en production. L'assistant de première mise en service est silencieusement contourné. Utilisez 1.1.0 ou plus récent ; les images antérieures à 1.1.0 seront retirées de Docker Hub pour éviter toute utilisation accidentelle.

1.2 Checklist de pré-installation

Avant de commencer :

1. Choisissez un répertoire hôte pour les fichiers mybibli (par exemple `/srv/mybibli` sur Linux, `/volume1/docker/mybibli` sur Synology DSM).

2. Choisissez comment stocker les images de couverture. Le fichier `docker-compose.yml` fourni déclare un *volume nommé* Docker (`mybibli_covers`) — aucune configuration, Docker gère la persistance et la sauvegarde via `docker volume`. Si vous préférez un répertoire visible dans votre gestionnaire de fichiers (File Station de Synology DSM, sauvegardes rsync manuelles), choisissez dès maintenant un chemin sur l'hôte (par exemple `/volume1/docker/mybibli/covers`) : vous basculerez le compose vers un bind mount à l'étape 3 plus bas.
3. Choisissez comment stocker les journaux (v1.7.0+). Le fichier `docker-compose.yml` fourni déclare un troisième volume nommé (`mybibli_logs`) pour les fichiers de journaux à rotation quotidienne. Comme pour les couvertures, vous pouvez garder le volume géré par Docker (sans configuration) ou basculer vers un bind mount visible côté hôte pour acheminer les journaux vers DSM Log Center, journald ou un collecteur externe. Voir section 1.3.
4. Choisissez un mot de passe fort pour les utilisateurs `root` et applicatif de la MariaDB embarquée — ou, pour une base de données externe, créez à l'avance une base dédiée et un utilisateur (voir section 1.4).

1.3 Méthode 1 — Base de données embarquée

C'est le chemin le plus simple. Le fichier `docker-compose.yml` livré déclare deux services : l'application `mybibli` et une instance MariaDB épinglée à une version connue. Les deux sont gérés comme une seule unité.

1. Récupérer le fichier compose et le `.env`

Téléchargez `docker-compose.yml` et `.env.example` depuis la version que vous comptez installer (ou clonez le dépôt sur le tag correspondant) :

```
mkdir -p /srv/mybibli && cd /srv/mybibli
curl -O https://raw.githubusercontent.com/guycorbaz/mybibli/v1.7.0/docker-compose.yml
curl -O https://raw.githubusercontent.com/guycorbaz/mybibli/v1.7.0/.env.example
cp .env.example .env
```

1b. (Optionnel) Basculer les couvertures vers un bind mount

Par défaut, le fichier `docker-compose.yml` stocke les images de couverture téléchargées dans le volume nommé Docker `mybibli_covers`. Cela fonctionne sans configuration et persiste à travers les mises à jour du conteneur (issue #213). La plupart des opérateurs peuvent laisser tel quel.

Si vous souhaitez voir les couvertures dans votre gestionnaire de fichiers (File Station de Synology DSM, sauvegardes rsync manuelles, etc.), éditez le fichier `compose` :

- Dans le bloc `volumes` : du service `mybibli`, commentez la ligne `- mybibli_covers:/app/covers` et décommentez la ligne `- ${COVERS_HOST_PATH:-./covers-host}:/app/covers` juste en-dessous.
- Dans `.env`, définissez `COVERS_HOST_PATH` sur le chemin hôte choisi dans la liste de vérification pré-installation (par exemple `COVERS_HOST_PATH=/volume1/docker/mybibli/covers`).
- Créez le répertoire avant de démarrer la pile (`mkdir -p`) pour éviter que Docker ne le crée automatiquement avec le propriétaire `root`.

1c. (Optionnel) Basculer les journaux vers un bind mount

Nouveau en **v1.7.0**. Par défaut, mybibli écrit ses fichiers de journaux à rotation quotidienne dans le volume nommé Docker mybibli_logs monté sur `/var/log/mybibli` dans le conteneur. Les fichiers sont nommés `mybibli.log.YYYY-MM-DD` et l'appendix en-process purge automatiquement les fichiers de plus de 30 jours.

Si vous souhaitez voir les journaux directement côté hôte (DSM Log Center, journald, agrégateur de logs externe, `tail` manuel), éditez le fichier compose :

- Dans le bloc `volumes` : du service mybibli, commentez la ligne `- mybibli_logs:/var/log/mybibli` et décommentez la ligne `- ${LOGS_HOST_PATH:-./logs-host}:/var/log/mybibli` juste en-dessous.
- Dans `.env`, définissez `LOGS_HOST_PATH` sur le chemin hôte voulu (par exemple `LOGS_HOST_PATH=/volume1/docker`).
- Créez le répertoire avant de démarrer la pile (`mkdir -p`) pour éviter que Docker ne le crée automatiquement avec le propriétaire root.

Le volume de journaux est uniquement médico-légal — vous pouvez le vider à tout moment sans affecter les données du catalogue. Voir chapitre 12 pour le suivi et le contrôle du niveau de journalisation.

2. Éditer .env

Ouvrez `.env` dans votre éditeur. Chaque variable porte un commentaire explicatif ; les valeurs qui comptent pour une installation avec base embarquée sont :

```

DATABASE_URL=mysql://mybibli:mybibli_password@db:3306/mybibli?charset=utf8mb4
MYSQL_HOST=db
MYSQL_DATABASE=mybibli
MYSQL_USER=mybibli
MYSQL_PASSWORD=<choisir-un-mot-de-passe-fort>
MYSQL_ROOT_PASSWORD=<choisir-un-mot-de-passe-fort>

HOST_PORT=8080          # à changer si 8080 est déjà pris sur l'hôte
APP_LANGUAGE=fr         # ou 'en' pour l'interface en anglais par défaut

MYBIBLI_COOKIE_SECURE=false # mettre à true uniquement derrière HTTPS

```

L'URL `DATABASE_URL` *doit* référencer les mêmes utilisateur, mot de passe et nom de base que les valeurs `MYSQL_*` — le fichier compose reconstruit l'URL à partir de ces parties mais le binaire Rust lit `DATABASE_URL` directement, les deux doivent donc concorder.

Attention

Les mots de passe par défaut dans `.env.example` (`mybibli_password`, `root_password`) sont des remplaçants, pas des identifiants à conserver. Changez-les avant le premier `docker compose up`. Une fois la base créée, les faire tourner nécessite aussi de réécrire les entrées utilisateur dans MariaDB — autant le faire correctement du premier coup.

3. Démarrer la pile

```

docker compose up -d
docker compose logs -f mybibli

```

Au premier lancement :

1. tire l'image `gcorbaz/mybibli:1.1.0` (ou ultérieure) et l'image MariaDB ;
2. démarre la base de données et attend qu'elle soit saine ;
3. exécute les migrations de schéma mybibli (initialisation unique) ;
4. démarre le serveur HTTP sur le port configuré.

Vous pouvez arrêter le suivi des logs avec `Ctrl-C` ; les conteneurs continuent de tourner en arrière-plan.

4. Ouvrir l'assistant de mise en service

Visitez <http://<hôte>:8080/setup> dans un navigateur. L'assistant de première mise en service apparaît (voir section 1.6).

1.4 Méthode 2 — Base de données externe (exemple NAS Synology)

Si vous exploitez déjà MariaDB sur votre NAS — par exemple via le paquet **MariaDB 10** de Synology — vous pouvez pointer mybibli dessus au lieu de faire tourner une seconde base dans Docker. L'image mybibli ne change pas ; seuls le fichier compose et le `.env` diffèrent.

1. Créer la base et l'utilisateur

Connectez-vous à la MariaDB externe en root et exécutez :

```
CREATE DATABASE mybibli CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'mybibli'@ '%' IDENTIFIED BY 'choisir-un-mot-de-passe-fort';
GRANT ALL PRIVILEGES ON mybibli.* TO 'mybibli'@ '%';
FLUSH PRIVILEGES;
```

Note

Sur Synology DSM, MariaDB 10 écoute sur le port 3307 par défaut (et non 3306 — ce port est réservé à l'ancien paquet MySQL). Ouvrez **MariaDB 10** → **Paramètres** et cochez *Activer la connexion TCP/IP* pour que les conteneurs sur le même pont Docker puissent atteindre le serveur.

2. Élaguer le fichier compose

Modifiez `docker-compose.yml` et retirez la base embarquée :

- supprimez (ou commentez) tout le service `db` ;
- supprimez (ou commentez) le bloc `depends_on` : sur le service mybibli ;
- dans le bloc `volumes` : en bas du fichier, retirez `mybibli_db_data` : uniquement — **conservez** `mybibli_covers` :. Les images de couverture sont locales à l'application et doivent survivre aux mises à jour du conteneur, peu importe où réside la base de données (issue #213).

3. Pointer `.env` vers la base externe

```
DATABASE_URL=mysql://mybibli:mot-de-passe-fort@host.docker.internal:3307/mybibli?charset=utf8mb4
```

Remplacez le nom d'hôte et le port qui correspondent à votre réseau. Depuis un conteneur mybibli qui tourne sur le même pont Docker que le NAS hôte, `host.docker.internal` résout vers l'hôte sur macOS et les installations Docker Linux récentes ; sur les moteurs Docker plus anciens, utilisez plutôt l'IP LAN du NAS (par exemple `192.168.1.10`).

Les variables `MYSQL_*` sont désormais inutilisées — laissez les vides ou supprimez-les.

4. Démarrer l'application

```
docker compose up -d
docker compose logs -f mybibli
```

Le premier lancement exécute les migrations de schéma sur la base externe. Le paquet MariaDB de Synology gardera ce schéma aussi longtemps que le paquet existe ; sa sauvegarde est couverte par les outils de backup du NAS (voir chapitre 7).

Astuce

La configuration avec base externe est le chemin recommandé lorsque vous maintenez déjà un serveur de bases de données, car elle vous donne un seul pipeline de sauvegarde à surveiller au lieu de deux. La configuration avec base embarquée est plus rapide à mettre en place si vous n'avez pas d'autre charge sur l'hôte.

1.5 Référence des variables d'environnement

La référence complète vit dans le fichier `.env.example` à la racine du dépôt — chaque variable porte un commentaire en ligne. Le tableau ci-dessous résume où chaque variable s'applique.

Toutes les variables booléennes suivent un **accept-set strict** : seules les chaînes `1`, `true` et `TRUE` comptent comme « activé ». Toute autre valeur (chaîne vide, `0`, `false`) est traitée comme « désactivé ». Cela évite le piège classique où une valeur shell éculée comme `0` se lit comme « positionnée » et bascule silencieusement un opt-out.

Variable	Défaut	Rôle
<i>Base de données</i>		
DATABASE_URL	— (requis)	Chaîne de connexion MariaDB (DSN).
MYSQL_HOST	db	Hôte pour le service DB embarqué.
MYSQL_PORT	3306	Port pour le service DB embarqué.
MYSQL_DATABASE	mybibli	Nom de base pour le service embarqué.
MYSQL_USER	mybibli	Utilisateur DB pour le service embarqué.
MYSQL_PASSWORD	—	Mot de passe utilisateur DB embarqué.
MYSQL_ROOT_PASSWORD	—	Mot de passe root pour le service embarqué.
<i>Serveur HTTP</i>		
HOST	0.0.0.0	Adresse de bind dans le conteneur.
PORT	8080	Port de bind dans le conteneur.
HOST_PORT	8080	Port publié par Docker sur l'hôte.
<i>Application</i>		
MYBIBLI_LOG_LEVEL	info	v1.7.0+. Niveau de journalisation à l'exécution. Niveau simple (<code>trace/debug/info/warn/error</code>) ou liste de directives <code>EnvFilter</code> . Prioritaire sur <code>RUST_LOG</code> . Aussi modifiable depuis <code>/admin > Système</code> .
MYBIBLI_LOG_DIR	/var/log/mybibli	v1.7.0+. Répertoire des fichiers de journaux à rotation quotidienne dans le conteneur. Mappé sur le volume nommé <code>mybibli_logs</code> .
LOGS_HOST_PATH	./logs-host	v1.7.0+. Chemin hôte optionnel pour bind mount ; utilisé uniquement si vous décommentez la ligne bind-mount dans <code>docker-compose.yml</code> .
RUST_LOG	mybibli=info	Filtre tracing hérité. Pris en compte si <code>MYBIBLI_LOG_LEVEL</code> n'est pas défini.
APP_LANGUAGE	en	Langue d'interface par défaut pour les visiteurs anonymes. v1.7.0 ajoute <code>de + it</code> à <code>en / fr</code> .
COVERS_DIR	./covers	Chemin de stockage des images de couverture.
COVERS_HOST_PATH	./covers-host	Chemin hôte optionnel pour bind mount ; utilisé uniquement si vous décommentez la ligne bind-mount dans <code>docker-compose.yml</code> .
<i>Durcissement</i>		
MYBIBLI_COOKIE_SECURE	false	Mettre à <code>true</code> uniquement derrière HTTPS.
CSP_REPORT_ONLY	false	Bascule l'en-tête CSP en mode report-only.
<i>Fournisseurs de métadonnées (migrés en DB au premier boot)</i>		
GOOGLE_BOOKS_API_KEY	—	Clé optionnelle pour Google Books.
OMDB_API_KEY	—	Clé optionnelle pour OMDb (films/séries).
TMDB_API_KEY	—	Clé optionnelle pour TMDb (films/séries).
<i>Overrides optionnels dev / test</i>		
MYBIBLI_SKIP_SETUP	false	Contourne l'assistant de première mise en service. <i>Jamais en prod.</i>
MYBIBLI_SKIP_STARTUP_PURGE	false	Saute l'auto-purge au démarrage.
MYBIBLI_SEED_DEV_USERS	false	Crée les utilisateurs dev <code>admin/admin + librarian/librarian</code> . <i>Jamais en prod.</i>

Les overrides d'URL de base des fournisseurs (`BNF_API_BASE_URL`, `GOOGLE_BOOKS_API_BASE_URL`, ...) n'existent que pour pointer le pipeline de métadonnées sur le serveur mock embarqué pendant les tests automatisés. Laissez-les vides en production.

1.6 Premier boot — l'assistant de mise en service

Sur une installation neuve, toute requête est redirigée vers `/setup` tant que l'assistant n'est pas terminé. L'assistant comporte quatre étapes :

1. **Compte administrateur.** Choisissez un nom d'utilisateur et un mot de passe fort pour le premier administrateur. Cette étape est toute la raison d'être de l'assistant — une fois le compte créé, le prédicat de la garde s'inverse et les visiteurs suivants atterrissent sur le catalogue public au lieu de `/setup`.
2. **Fournisseurs de métadonnées.** Collez les clés d'API que vous avez sous la main (Google Books, OMDb, TMDb). Tous les fournisseurs à clé sont optionnels — vous pouvez sauter cette étape et ajouter les clés plus tard depuis `/admin > System > Metadata Providers`.
3. **Préférences.** Choisissez la langue d'interface par défaut pour les visiteurs anonymes et le seuil de retard de prêt (en jours).
4. **Terminé.** Un écran récapitulatif. Après confirmation, l'assistant écrit `settings.setup_completed_at` et se désactive pour la durée de vie de l'installation.

L'assistant est idempotent : si vous fermez le navigateur entre les étapes, rouvrez `/setup` et l'assistant reprend côté serveur à la bonne étape (il interroge la base à chaque chargement — il n'y a pas d'état client à perdre).

1.7 Vérification de l'installation

Après la fin de l'assistant :

- Connectez-vous sur <http://<hôte>:8080/login> avec les identifiants admin que vous venez de créer.
- Visitez `/admin > Health`. La page doit afficher des compteurs d'entités non-nuls (zéro sur une installation neuve — c'est normal) et une coche verte à côté de chaque fournisseur de métadonnées qui a été joignable dans les cinq dernières minutes.
- Scannez un ISBN (le champ de recherche en haut de la page d'accueil) pour confirmer que le pipeline de métadonnées résout bien un vrai titre. Le premier scan prend quelques secondes ; le résultat apparaît dans la timeline des ajouts récents sur la page d'accueil une fois la récupération d'arrière-plan terminée.

Si quelque chose tourne mal, sautez au chapitre [10](#).

Chapitre 2

Configuration

Une fois l’assistant de mise en service passé, le travail de configuration du jour 1 se compose de trois blocs : définir vos *lieux de stockage*, peupler les *données de référence* (genres, rôles de contributeur, états de volume, types de nœud de localisation) et ajuster les *paramètres globaux* (seuil de retard, cadence d’auto-purge, clés de fournisseurs).

Tout se configure depuis le panneau `/admin` — il n’y a pas de fichier de configuration séparé. Le panneau a cinq onglets :

- **Health** — tableau de bord en lecture seule.
- **Users** — créer/désactiver des utilisateurs, changer de rôle.
- **Reference data** — les quatre tables taxonomiques.
- **Trash** — voir et restaurer les éléments soft-deleted.
- **System** — paramètres globaux.

Seul le rôle `Admin` voit le panneau ; les utilisateurs `Librarian` et `Anonymous` reçoivent un 403 sur toute route `/admin/*`.

2.1 Lieux de stockage et hiérarchie

Les lieux de stockage sont les endroits physiques où vivent vos volumes : une pièce, une étagère, une rangée, une boîte numérotée, une case précise d’une armoire. mybibli les organise comme un **arbre** — chaque lieu a au plus un parent. La hiérarchie est arbitraire : vous décidez à la fois des niveaux et des libellés.

Une hiérarchie domestique exploitable peut tenir sur trois niveaux :

Salon	(pièce)
-- Bibliothèque A	(étagère)
-- Rangée 1	(rangée)
-- Rangée 2	(rangée)
-- Bibliothèque B	(étagère)
-- Rangée 1	(rangée)
Chambre	(pièce)
-- Table de nuit	(étagère)

Trois niveaux suffisent à retrouver n'importe quel volume en quelques secondes sans forcer la hiérarchie à épouser chaque recoin de votre intérieur. Une liste plate (une grosse « étagère » par étagère physique) fonctionne tout aussi bien pour des collections de quelques centaines de volumes.

Éditer l'arbre

Ouvrez `/locations`. L'arbre est rendu sous forme de liste imbriquée avec des actions création / renommage / déplacement / suppression en ligne. Le glisser-déposer *n'est pas* pris en charge (mybibli applique une CSP stricte sans JavaScript inline) ; les déplacements se font en éditant un nœud et en choisissant un nouveau parent dans un menu déroulant.

Types de nœud de localisation

Chaque nœud de localisation porte un **type de nœud** — « pièce », « étagère », « rangée », « boîte », ce que vous voulez. Les types sont gérés depuis `/admin > Reference data > Location node types`. Ce sont de purs libellés ; mybibli n'impose pas qu'une « étagère » soit sous une « pièce ».

Astuce

Renommer un type de nœud propage la modification à chaque lieu qui l'utilise (le type est stocké par nom dans la table des lieux). Supprimer un type est refusé tant qu'au moins un lieu y fait référence.

2.2 Étiquettes : codes V et codes L

mybibli imprime deux types d'étiquettes à code-barres :

- **Codes V** — un par *volume* (l'exemplaire physique sur l'étagère). Format : V suivi d'un numéro zéro-paddé (par exemple V0042).
- **Codes L** — un par *lieu de stockage*. Format : L suivi d'un numéro zéro-paddé (par exemple L0007).

Les codes V et L sont scannés avec la même douchette code-barres que vous utilisez pour les ISBN au moment du catalogage. Le préfixe (V ou L) indique à mybibli quel lookup effectuer.

Imprimer les étiquettes

Ouvrez `/locations` ou la page du volume concerné et cliquez **Imprimer l'étiquette**. mybibli rend une page HTML imprimable avec le code-barres Code-128 et le texte lisible. Imprimez sur papier ordinaire ou sur planches autocollantes — l'encodage est suffisamment robuste pour qu'une imprimante jet d'encre bas de gamme suffise pour les codes V comme les codes L.

Astuce

Imprimez vos étiquettes de codes V *avant* de commencer un catalogage en série. Une planche pré-imprimée de V0001 à V0500 permet de saisir un autocollant, de le coller au dos du livre et de scanner d'un même mouvement — sans aller-retour entre l'interface de catalogage et l'imprimante.

2.3 Matériel : douchette code-barres

mybibli est conçu autour d’une douchette code-barres mais n’en exige pas strictement une : tous les champs ISBN, code V et code L acceptent la saisie clavier comme repli. Cela dit, dès que la collection dépasse une poignée de titres, une douchette USB bon marché est rentabilisée après la première douzaine de scans.

Quoi acheter

N’importe quelle douchette USB en *mode clavier HID* dans la gamme 15–30 € convient. Le mode HID est ce qui rend la douchette sans pilote : l’OS la voit comme un clavier ordinaire et les chiffres scannés atterrissent directement dans le champ qui a le focus. Presque toutes les douchettes USB grand public sortent d’usine dans ce mode ; cherchez les mots “HID”, “émulation clavier” ou “plug-and-play” sur la fiche produit.

La douchette doit prendre en charge les deux symbologies que mybibli utilise :

- **EAN-13** — le code à 13 chiffres au dos des livres, CD, DVD et de la plupart des produits grand public. Universellement supporté.
- **Code-128** — la symbologie utilisée par mybibli pour encoder les codes V et codes L sur les étiquettes qu’il imprime (voir section 2.2). Universellement supporté.

Vous n’avez *pas* besoin d’un imageur 2D (le type qui lit les QR codes) — une douchette laser ou LED 1D basique suffit. mybibli n’utilise pas les QR codes.

Configuration

Trois réglages comptent côté douchette. La plupart sortent d’usine correctement configurées ; si vos scans se comportent bizarrement, consultez le manuel de la douchette ou la planche de codes-barres de configuration livrée dans la boîte :

- **Termineur / suffixe : Enter** (parfois appelé CR ou Carriage Return). Le formulaire de catalogage de mybibli soumet automatiquement à la pression d’Entrée, donc ce réglage transforme chaque scan en un seul aller-retour. Toute autre valeur (Tab, pas de termineur) ne marchera pas sans intervention clavier manuelle.
- **Préfixe** : aucun. Certaines douchettes peuvent préfixer une chaîne fixe au début du payload — laissez cette option désactivée.
- **Disposition clavier** : *doit* correspondre à la disposition de l’hôte sur lequel tourne le navigateur. Si votre ordinateur utilise AZERTY mais la douchette est configurée en QWERTY, les chiffres sortent en lettres (p. ex. 1234567890 arrive comme &é''' (-è_çà) et aucun ISBN ne résout. Le manuel de la douchette contient une planche de codes pour changer la disposition.

Astuce

Douchettes USB sur NAS Synology : peu importe que la douchette soit branchée sur l’ordinateur portable depuis lequel vous accédez à l’interface web de mybibli plutôt que sur le NAS lui-même. La douchette est un clavier pour le navigateur, pas pour le serveur. Le même montage marche avec une appli scanner sur téléphone, une douchette Bluetooth industrielle appairée à l’ordinateur, ou n’importe quelle douchette USB clé.

Quand la douchette n'est pas disponible

Chaque champ de scan accepte aussi la saisie clavier. Cliquez dans le champ étiqueté “Scanner ISBN” (ou “Scanner code V” / “Scanner code L” ailleurs), tapez les chiffres, pressez Entrée. Le formulaire se comporte à l'identique.

2.4 Données de référence

Quatre tables taxonomiques pilotent les menus déroulants que vous voyez pendant le catalogage. Elles vivent sous `/admin > Reference data` :

Genres Les genres assignés aux titres. mybibli livre un seed de base FR/EN (Fiction, Documentaire, Biographie, ...) — éditez ou étendez pour coller à votre collection.

Rôles de contributeur Auteur, Illustrateur, Traducteur, Éditeur, Narrateur, ... Utilisés sur les pages détail de titre pour attribuer les bonnes personnes aux bons rôles.

États de volume L'état physique d'un exemplaire — Neuf, Bon, Usé, Endommagé, ... Utile pour les retours de prêt (marquer un volume Usé quand un emprunteur l'a abîmé).

Types de nœud de localisation Voir section 2.1.

Sémantique CRUD

Les quatre tables partagent le même schéma CRUD :

- **Créer** ajoute une ligne. Si le nom entre en collision avec une ligne soft-deleted de la même table, la ligne existante est réactivée et le résultat affiche *Réactivé* au lieu de *Créé*.
- **Renommer** met à jour le nom. Pour les types de nœud de localisation, le nouveau nom cascade sur tous les lieux qui utilisent ce type.
- **Supprimer** effectue un soft-delete *uniquement si la ligne est inutilisée*. Si un titre référence encore ce genre / rôle / état / type, la suppression est refusée avec un comptage des dépendants.

Les données de référence ne sont pas traduites

Les valeurs des données de référence sont stockées telles quelles — le seed FR crée un genre appelé « Roman » et c'est ce texte littéral que verront les utilisateurs anglophones s'ils basculent l'interface en anglais. Seule l'ossature de l'interface (libellés de boutons, navigation, messages d'erreur) est traduite.

2.5 Paramètres système

L'onglet `/admin > System` regroupe tous les paramètres globaux. Le panneau est divisé en trois formulaires, chacun avec son bouton de sauvegarde :

Prêts

- **Seuil de retard (jours)**. Définit combien de jours un prêt peut courir avant d'apparaître comme en retard sur le tableau de bord. Défaut : 30.
- **Timeout d'inactivité de session (secondes)**. Temps d'inactivité après lequel une session authentifiée est effacée. Défaut : 1800 s (30 minutes).

- **Intervalle d'auto-purge (secondes).** Cadence à laquelle le scheduler d'arrière-plan supprime définitivement les éléments soft-deleted depuis plus de 30 jours. Défaut : 86400 s (24 heures).

Fournisseurs de métadonnées

Une ligne par fournisseur à clé (Google Books, OMDb, TMDb). Chaque ligne a un champ clé et un bouton *Effacer*. Sauvegarder une clé prend effet à la prochaine récupération de métadonnées — pas de redémarrage, pas de redéploiement.

Note

Si la même clé de fournisseur est positionnée dans `.env` et effacée depuis l'interface, le prochain reboot re-migre la valeur depuis `.env`. L'intention de l'opérateur au moment du déploiement l'emporte au boot. Pour rendre l'effacement durable, retirez aussi la variable de `.env` (ou de `docker-compose.yml`).

Langue

Fixe la langue d'interface par défaut pour les visiteurs *anonymes*. Les utilisateurs authentifiés outrepassent ce choix via le bouton bascule dans la barre de navigation — leur choix est stocké sur leur ligne utilisateur.

2.6 Utilisateurs

Ouvrez `/admin > Users` pour gérer les comptes utilisateur. Chaque ligne montre le nom d'utilisateur, le rôle, le drapeau actif et la date de dernière activité. Les actions disponibles :

- **Créer un utilisateur** (haut de page) — nom, mot de passe, rôle.
- **Éditer** — changer le rôle ou renommer. Ne change pas le mot de passe d'un autre utilisateur depuis cette vue ; les changements de mot de passe passent par la page `/account` de l'utilisateur lui-même.
- **Désactiver** — soft-delete l'utilisateur. Ses sessions sont effacées immédiatement ; il ne peut plus se reconnecter.
- **Réactiver** — défait une désactivation (depuis l'onglet `Trash`).

Deux garde-fous s'appliquent :

- **L'auto-désactivation est bloquée.** Vous ne pouvez pas désactiver le compte avec lequel vous êtes connecté.
- **Garde du dernier admin actif.** Le système maintient toujours au moins un admin actif. Désactiver le dernier est refusé avec un 409 et un message d'erreur clair.

Les trois rôles (Anonyme, Bibliothécaire, Administrateur) sont détaillés au chapitre 6.

Chapitre 3

Usage au quotidien

Ce chapitre parcourt les flux que vous lancerez des dizaines de fois une fois la bibliothèque opérationnelle : cataloguer un nouveau volume, chercher dans la collection, déplacer un volume, le prêter, le récupérer, et auditer ce qui est sur les étagères.

3.1 La page d'accueil

La page d'accueil est le tableau de bord. De haut en bas :

- **Champ de scan.** Un champ focalisé qui accepte un ISBN, un code V ou un code L. La couche scanner-guard route l'entrée vers la bonne page selon le préfixe.
- **Bandeau d'indicateurs.** Ajouts récents, prêts en retard, séries avec lacunes, volumes non rangés — chaque clic ouvre la liste filtrée correspondante.
- **Activité récente.** Les sept derniers jours de catalogage et de retours.

3.2 Cataloguer un nouveau titre via code-barres

1. Focalisez le champ de scan de la page d'accueil (il est sélectionné par défaut au chargement). Scannez l'ISBN / EAN-13 du dos de l'ouvrage.
2. mybibli redirige vers la page de catalogage, qui affiche un titre squelette avec le code-barres scanné. Une tâche d'arrière-plan lance des requêtes contre la chaîne de fournisseurs de métadonnées.
3. En quelques secondes, le squelette se remplit : titre, contributeurs, éditeur, année, image de couverture. Un petit indicateur confirme quel fournisseur a résolu le titre.
4. Choisissez un **lieu** dans le menu déroulant (ou scannez le code L du rayon de destination) et un **état de volume** (typiquement *Neuf*). Cliquez **Ajouter un volume**.
5. Imprimez et collez une étiquette de code V sur le volume.

Si le pipeline de métadonnées échoue à résoudre l'ISBN, le titre apparaît avec un badge *Non résolu*. Vous pouvez modifier les champs manuellement et le volume est créé malgré tout.

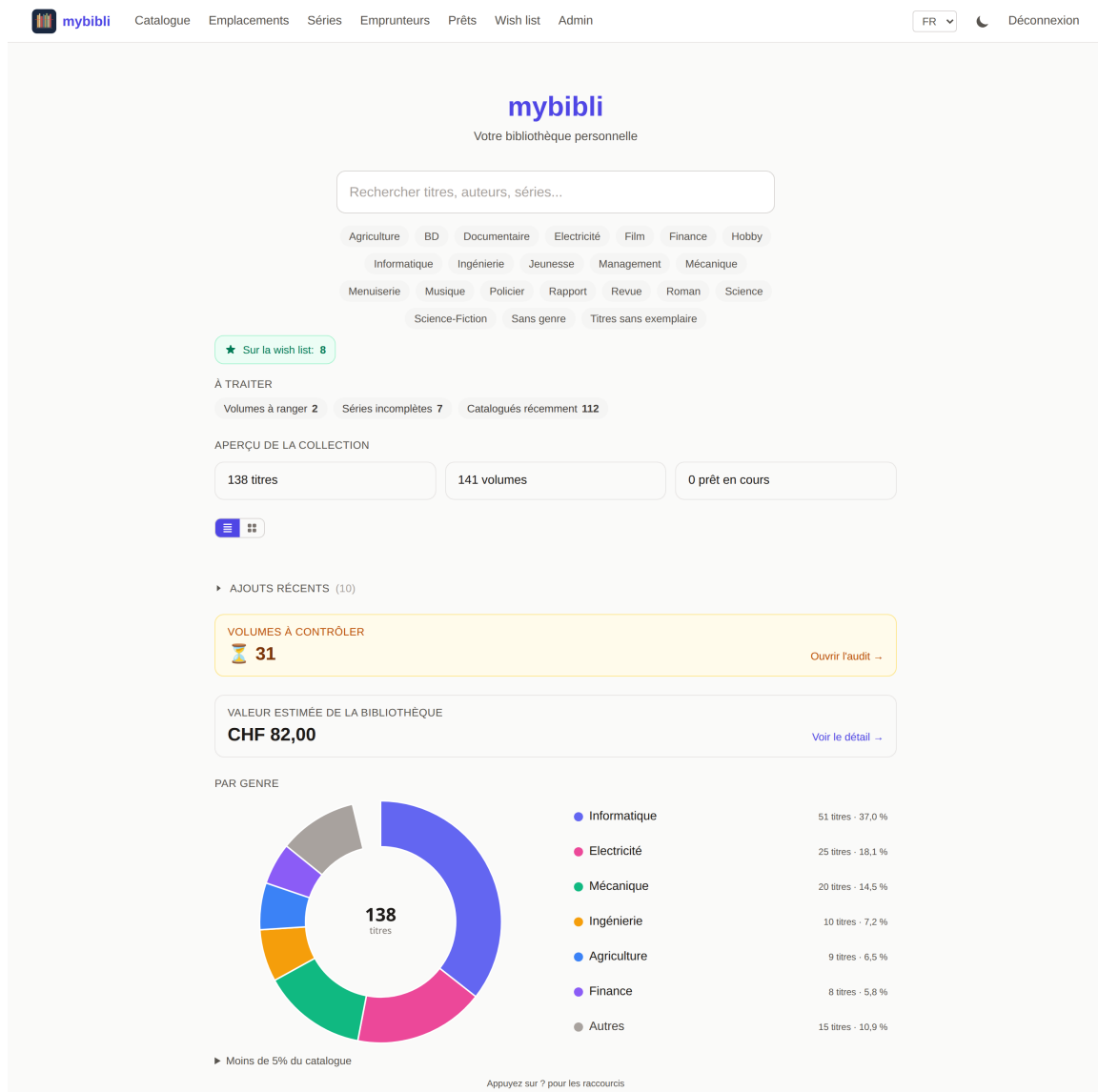


Fig. 3.1 : La page d'accueil sur une installation réelle : champ de recherche, puces de filtres par genre, compteurs « À traiter », totaux « Aperçu de la collection » et bandeau d'ajouts récents avec les jaquettes récupérées via la chaîne de fournisseurs de métadonnées.

Astuce

Le catalogage en série marche le mieux quand la planche de codes V, la douchette et l'imprimante sont à portée de main. La plupart du temps est consacré à scanner, pas à attendre mybibli — la récupération de métadonnées est asynchrone, vous n'avez donc pas à attendre qu'elle se termine avant de passer au volume suivant.

3.3 Rechercher dans la collection

La page `/search` accepte :

- **titre** complet ou partiel ;
- **nom de contributeur** (matche l'auteur par défaut ; les autres rôles de contributeur sont

scorés moins fort) ;

- **ISBN / EAN-13** — match exact ;
- **code V** ou **code L** — match exact ;
- **code Dewey** (pour les bibliothèques qui utilisent Dewey).

Les résultats sont triés par pertinence. Cliquez un résultat pour ouvrir la page du titre ; cliquez un contributeur pour filtrer le catalogue sur ses œuvres ; cliquez le libellé d'un lieu pour voir chaque volume actuellement à cet endroit.

Astuce

Vous pouvez « rechercher » un seul volume en scannant son code V depuis n'importe quelle page qui dispose du champ de scan d'accueil — y compris la page d'accueil, `/search` et `/loans`. Plus rapide que la frappe.

3.4 Déplacer un volume vers un autre lieu

1. Ouvrez la page du volume (recherche par code V ou clic depuis la page du titre).
2. Cliquez **Déplacer**.
3. Scannez le code L du lieu de destination, ou choisissez-le dans le menu déroulant.
4. Confirmez.

mybibli enregistre le déplacement dans l'historique de lieu du volume, vous permettant de retracer où un livre a vécu au fil du temps.

3.5 Prêts

Prêter un volume

1. Ouvrez `/loans` et cliquez **Nouveau prêt**.
2. Scannez ou choisissez l'**emprunteur**. Si l'emprunteur n'existe pas encore, cliquez *Créer l'emprunteur* en ligne.
3. Scannez ou choisissez le **volume** (code V).
4. Réglez optionnellement une **date de retour prévue** ; sinon mybibli la calcule à partir du seuil de retard (chapitre 2).
5. Confirmez.

Le lieu actuel du volume est inscrit sur la ligne de prêt pour qu'un retour ultérieur puisse le restaurer au bon rayon.

Retourner un volume

1. Ouvrez `/loans` et trouvez le prêt actif (filtre *Actifs* ou scan du code V).
2. Cliquez **Retourner**.
3. Choisissez l'état post-retour (*Bon, Usé, ...*).

4. Confirmez. Le volume retourne automatiquement à son lieu d'avant prêt.

Prêts en retard

Les prêts qui dépassent le seuil de retard passent au rouge sur la page `/loans` et sur l'indicateur *En retard* de la page d'accueil. `/loans?filter=overdue` donne une liste focalisée.

3.6 Audit de la bibliothèque

Un *audit* c'est le rituel « je parcours les étagères et je confirme que tout est bien à la place que la base indique ». mybibli ne livre pas un mode audit dédié — le flux équivalent est :

1. Filtrez `/search` par le lieu que vous comptez auditer, trié par code V.
2. Parcourez le rayon en scannant chaque code V.
3. Pour tout volume scanné qui *n'est pas* dans la liste filtrée, mybibli affiche un message *Volume absent de ce lieu*. Déplacez-le (section 3.4) et continuez.
4. Tout volume *dans* la liste filtrée que vous *n'avez pas* scanné après avoir parcouru le rayon est suspect — soit déplacé, soit manquant. Marquez-le pour suivi.

Astuce

Un audit trimestriel attrape les volumes mal rangés tôt — plus un livre traîne au mauvais rayon, plus il est difficile de se rappeler où il aurait dû être.

3.7 Rangement après retours

Après une journée de retours, les volumes s'empilent près du poste de scan. Le flux de *rangement* de mybibli accélère la remise en rayon :

1. Prenez un volume et scannez son code V.
2. mybibli affiche la page du volume avec le lieu d'origine bien en évidence.
3. Portez le volume au rayon indiqué. Scannez le code L du rayon pour confirmer.
4. Recommencez.

La confirmation par code L est optionnelle mais recommandée — elle laisse une entrée fraîche dans l'historique du volume sur laquelle le prochain audit pourra s'appuyer.

3.8 Wish list

La *wish list* est une liste séparée des livres que vous aimeriez acheter, distincte du catalogue possédé. Elle vit sous `/wishlist` (voir la barre de navigation) et est disponible pour les Bibliothécaires et les Admins.

3.8.1 Ajouter une entrée

Le formulaire *Ajouter à la wish list* propose deux modes :

- **Par ISBN.** Scannez ou saisissez un ISBN à 13 chiffres. mybibli interroge la même chaîne de fournisseurs de métadonnées que le catalogue, puis affiche une carte de prévisualisation (titre, auteur, éditeur, année, couverture). Cliquez sur *Ajouter à la wish list* pour enregistrer. Les images de couverture pointent directement sur l'URL du fournisseur résolu (Open Library Covers ou Google Books par exemple) ; le téléchargement local de la couverture est sur la roadmap.
- **Saisie libre.** À utiliser quand vous n'avez pas d'ISBN sous la main (titre auto-édité, recommandation notée pendant le déjeuner). Seul le *Titre* est obligatoire ; auteur, éditeur et notes sont optionnels.

3.8.2 Parcourir en librairie

La vue liste se transforme en pile de cartes sur les petits écrans — conçue pour parcourir rapidement d'un pouce dans une librairie. Chaque carte affiche le titre, l'auteur (s'il est connu), l'ISBN (s'il y en a un), l'éditeur et la date d'ajout.

Pour imprimer une copie papier ou sauver un PDF pour usage hors-ligne, cliquez sur *Imprimer* en haut de la liste. mybibli rend une page d'impression épurée ; utilisez *Imprimer en PDF* du navigateur (Fichier → Imprimer → Enregistrer en PDF, ou Ctrl/Cmd-P) pour générer un PDF à emporter à la librairie.

3.8.3 Marquer un livre comme acheté

Quand vous achetez un livre, ouvrez son entrée wish list (ou tapez *Acheté* directement depuis la ligne de la liste), confirmez dans la modale, et l'entrée est mise en soft-delete. L'action est réversible depuis le panneau Corbeille de l'admin pendant 30 jours.

3.8.4 Auto-détection au scan catalogue

Quand vous cataloguez un livre fraîchement acheté via le flux standard de scan (*/catalog*), mybibli vérifie si son ISBN correspond à une entrée wish list active. Si oui, un bandeau émeraude apparaît dans la zone de feedback : *Cet ISBN est sur votre wish list — la retirer ?* Un clic sur le bouton *Retirer de la wish list* ouvre la même modale *Marquer comme acheté*, et la confirmation supprime l'entrée. Aucune suppression silencieuse.

Chapitre 4

Types de médias multiples

mybibli n'est pas réservé aux livres. Quatre types de médias sont des citoyens de première classe :

- **Livre** — le défaut. Romans, essais, ouvrages de référence, albums illustrés pour enfants.
- **BD / comic** — comic individuel ou album omnibus multi-positions (par exemple une « anthologie Tintin » qui réunit physiquement les volumes 1 à 5).
- **Audio** — CD, vinyles, livres audio sur support physique.
- **Film / série** — DVD, coffrets Blu-ray, intégrales de séries TV.

Le type de média pilote trois choses :

1. **Quel fournisseur de métadonnées est consulté en premier.** Les ISBN routent vers BnF + Google Books + Open Library ; les films routent vers TMDb + OMDb ; l'audio route vers MusicBrainz.
2. **Quels champs sont exposés sur la page du titre.** Les titres audio font apparaître une track list ; les films font apparaître réalisateur, durée, restriction d'âge.
3. **L'icône affichée à côté du titre** dans les résultats de recherche et la liste de catalogue.

4.1 Livres

Le type de média par défaut. Catalogage tel que décrit en section 3.2. Les séries multi-volumes (encyclopédies, trilogies, ...) sont modélisées par une entité *série* au chapitre 2.

4.2 BD et comics

Un volume **omnibus** peut contenir plusieurs positions dans une série. mybibli gère cela via la fonctionnalité *multi-position* : au moment de la création d'un volume, vous pouvez fixer la *plage de positions* (par exemple « contient les volumes 5 à 7 »). La détection de lacunes de série (chapitre 3) comprend ces plages et ne signalera pas les positions 5, 6 ou 7 comme manquantes une fois l'omnibus en rayon.

4.3 Sorties audio

Les titres audio utilisent MusicBrainz comme source principale de métadonnées. Le code-barres au dos d'un CD (EAN-13) se résout de la même façon qu'un ISBN.

- **Pistes.** MusicBrainz renvoie la liste des pistes ; mybibli les affiche sur la page du titre.
- **Support.** CD vs vinyle vs cassette est capturé sur la ligne du volume (pas sur le titre) — deux pressages identiques sur supports différents sont deux volumes du même titre.

4.4 Films et séries

Les films et séries TV utilisent OMDb et TMDb comme sources principales de métadonnées. Un coffret de série est modélisé comme un *titre* avec un volume multi-position couvrant la plage de disques.

Astuce

OMDb est gratuit pour de faibles volumes (moins de 1000 requêtes par jour) ; TMDb est gratuit avec attribution. L'un ou l'autre couvre largement une étagère DVD / Blu-ray familiale. Voir le chapitre 5 pour la mise en place.

4.5 Choisir le type de média au scan

Après scan d'un code-barres inconnu, mybibli devine le type de média depuis le préfixe EAN (le préfixe pays GS1 et la réponse du fournisseur). La devinette est affichée sur la page de catalogage ; vous pouvez la corriger avant de cliquer **Ajouter un volume**. Re-typing un titre plus tard se fait en quelques clics (`/title/<id>`) et la chaîne de fournisseurs ré-exécute contre le nouveau type si vous déclenchez *Re-fetch metadata*.

Chapitre 5

Fournisseurs de métadonnées

Lorsque vous scannez un ISBN ou un EAN-13, mybibli ne cherche pas le titre dans un catalogue local — il interroge un ou plusieurs *fournisseurs de métadonnées* externes jusqu'à ce que l'un d'eux renvoie une réponse propre. Ce chapitre explique la chaîne, où récupérer les clés d'API et comment les faire tourner en toute sécurité.

5.1 La chaîne de fournisseurs

mybibli parcourt les fournisseurs dans l'ordre le mieux adapté au type de média deviné :

Fournisseur	Types de média	Notes	Clé ?
BnF	Livres	Catalogue de la Bibliothèque nationale de France ; idéal pour les titres en français.	non
Open Library	Livres	Catalogue bénévole ; large couverture anglaise / multilingue.	non
Google Books	Livres	Commercial ; bon sur les essais grand public.	optionnelle
MusicBrainz	Audio	Métadonnées musicales bénévoles ; très complet sur CD / vinyles.	non
OMDb	Films, séries	Palier gratuit ; quotas.	requis
TMDb	Films, séries	Gratuit avec attribution.	requis
BDGest	BD, comics	Catalogue BD francophone (web-scraped).	non

Le premier fournisseur qui renvoie un enregistrement *propre* l'emporte. « Propre » veut dire : titre présent, au moins un contributeur, et une année. Les correspondances partielles sont enregistrées mais mybibli continue à parcourir la chaîne jusqu'à trouver une correspondance propre ou épuiser les fournisseurs.

Repli sur les couvertures

Quand le fournisseur qui résout le titre renvoie ses métadonnées mais aucune URL de couverture (la BnF ne porte jamais d'URL d'image dans son UNIMARC ; Google Books n'a parfois pas

d'`imageLinks` pour les titres auto-édités), mybibli se rabat sur l'API *Open Library Covers* (<https://covers.openlibrary.org/>) indexée par ISBN. Cet index de couvertures est large et indépendant de l'exhaustivité du catalogue Open Library, donc la plupart des livres récupèrent une jaquette même si leurs métadonnées textuelles proviennent d'un autre fournisseur. Le repli ne s'enclenche que lorsque (a) la chaîne a réussi *et* (b) le fournisseur résolvant n'a pas fourni de `cover_url`, de sorte qu'un titre dont aucun fournisseur n'a la couverture finit sans image, sans rafale d'essais inutiles.

5.2 Clés d'API

Trois fournisseurs requièrent une clé d'API :

Google Books

Optionnelle mais relève significativement le budget de requêtes quotidien. Sans clé, le fournisseur Google Books reste désactivé — la chaîne fonctionne toujours (BnF → Open Library n'exigent pas de clé), mais les ouvrages anglophones grand public que BnF ne référence pas arriveront avec des métadonnées vides.

Le parcours dans la Google Cloud Console déroute souvent les nouveaux venus parce que l'étape de création de la clé est nichée à deux menus de profondeur. Suivez exactement ces étapes :

1. Ouvrez <https://console.cloud.google.com/> et connectez-vous.
2. Sélecteur de projet en haut (à côté du logo *Google Cloud*) → *Nouveau projet* → nommez-le librement (p.ex. `mybibli-metadata`) → *Créer*. Attendez quelques secondes que le projet soit sélectionné.
3. Ouvrez le menu hamburger (en haut à gauche, trois lignes horizontales) → *APIs & services* → *Bibliothèque*. Cherchez **Books API**, cliquez dessus, puis *Activer*.
4. Toujours sous *APIs & services*, ouvrez *Identifiants*. Cliquez + *Créer des identifiants* → *Clé d'API*. Une fenêtre affiche une clé fraîchement générée — copiez-la maintenant.
5. Recommandé : cliquez *Restreindre la clé* → sous *Restrictions d'API* choisissez *Restreindre la clé* → cochez uniquement *Books API*. Laissez *Restrictions d'application* sur *Aucune* (mybibli n'envoie pas d'en-tête *Referer*). Enregistrez.
6. Dans mybibli : `/admin > System > Metadata Providers` → collez dans le champ **Google Books API key** → *Enregistrer*. Le prochain scan la prend en compte — pas de redémarrage requis.

Pour vérifier que la clé est active, scannez un ISBN anglophone que BnF ne connaît pas (*Effective Java*, 9780134685991, est un bon canari). Les métadonnées doivent se peupler en quelques secondes. Si elles restent vides, voyez *Dépannage* en fin de chapitre.

OMDb

1. Visitez <https://www.omdbapi.com/apikey.aspx>.
2. Choisissez le palier gratuit (1000 requêtes/jour).
3. Confirmez via le courriel d'activation reçu.
4. Collez la clé dans mybibli.

TMDb

1. Inscrivez-vous sur <https://www.themoviedb.org/>.
2. Ouvrez *Settings* → *API* et demandez une clé pour *usage personnel / non-commercial*.
3. Collez la clé dans mybibli.

5.3 Où vivent les clés

Les clés transitent par deux couches :

1. **Variables d'environnement** (`GOOGLE_BOOKS_API_KEY`, `OMDB_API_KEY`, `TMDB_API_KEY`). Elles sont migrées *une fois* au boot dans la table `settings` — elles servent essentiellement d'amorçage au moment du déploiement.
2. **La table `settings`**, exposée via `/admin > System > Metadata Providers`. C'est la source de vérité au moment de la requête. La rotation d'une clé depuis l'interface prend effet à la toute prochaine récupération de métadonnées.

L'implication est asymétrique :

- Si vous positionnez une clé uniquement en variable d'environnement, le premier boot la copie dans la base et ensuite c'est la base qui fait foi.
- Si vous positionnez une clé uniquement dans l'interface, la variable d'environnement n'est plus jamais consultée.
- Si vous positionnez les deux puis *Effacez* la clé depuis l'interface, la valeur en base devient vide — mais au boot suivant, la valeur d'environnement y est re-migrée. Pour rendre l'effacement durable, retirez aussi la variable d'environnement de `.env / docker-compose.yml`.

5.4 Santé des fournisseurs

`/admin > Health` pingue chaque fournisseur enregistré toutes les cinq minutes et rapporte son état sous forme de coche verte ou de croix rouge. Une croix rouge ne signifie pas toujours que le fournisseur est en panne — cela peut aussi vouloir dire que la clé d'API est absente ou invalide. L'onglet Health inclut le timestamp du dernier check et le code HTTP pour faciliter le diagnostic.

5.5 Quotas et back-pressure

mybibli applique un petit quota par fournisseur (quelques requêtes par seconde). Cataloguer cent volumes à la suite ne déclenchera pas de 429 distant. Si un fournisseur renvoie quand même 429, la requête est loguée et le fournisseur suivant de la chaîne est consulté — le flux de catalogage ne se bloque jamais sur un fournisseur en quota.

Attention

Ne committez jamais une vraie clé d'API dans git. Le fichier `.env` est dans `.gitignore` pour cette raison ; `.env.example` ne porte que des placeholders. Si une clé fuit, faites-la tourner immédiatement sur la console du fournisseur — coller une nouvelle clé dans mybibli prend effet en quelques secondes.

5.6 Dépannage : métadonnées vides

Si un ISBN fraîchement scanné finit avec des champs titre / auteur / description vides, parcourez cette liste avant de suspecter un bug :

1. **État des fournisseurs dans /admin > Health.** Si Google Books ou Open Library affichent une croix rouge, ces fournisseurs sont sautés à chaque récupération. Vérifiez le code HTTP du dernier check — 403 ou 401 signifie en général une clé d'API absente ou invalide, pas une panne du fournisseur.
2. **Langue du titre.** BnF est le fournisseur primaire et privilégie le français. Pour les livres anglophones, la chaîne bascule sur Google Books — donc la clé (voir plus haut) est la pièce qui débloque la situation.
3. **Format de l'ISBN.** mybibli normalise EAN-13 vs ISBN-10 en interne, mais certains fournisseurs n'indexent que l'un des deux. Un second scan quelques minutes plus tard frappe parfois une route de repli différente.
4. **Édition manuelle.** Vous pouvez toujours saisir titre, genre, contributeurs etc. à la main depuis la page de détail du titre — la sauvegarde fonctionne même si aucun fournisseur n'a répondu. Le genre `Non classé` est la valeur par défaut pour un titre arrivé sans données de classification.

Si les champs restent vides après avoir configuré les bonnes clés et attendu quelques secondes, ouvrez une issue sur le dépôt du projet avec l'ISBN qui échoue — les mainteneurs ajoutent des mocks de fournisseurs pour les cas reproductibles.

Chapitre 6

Rôles et permissions

mybibli a trois rôles utilisateur, ordonnés par privilège :

1. **Anonyme** — quiconque arrive sur le site sans cookie de session. Accès en lecture seule au catalogue. Ne peut ni prêter, ni éditer, ni supprimer.
2. **Bibliothécaire** — utilisateur authentifié de rôle `librarian`. Catalogage au quotidien, prêts, retours, déplacements de lieu. Ne peut ni gérer les utilisateurs ni changer les paramètres système.
3. **Administrateur** — utilisateur authentifié de rôle `admin`. Tout ce qu'un bibliothécaire peut faire, plus le panneau `/admin` : utilisateurs, données de référence, corbeille, paramètres système.

6.1 Qui peut faire quoi

Action	Anon	Bibl.	Admin
Parcourir / rechercher dans le catalogue	oui	oui	oui
Voir les pages de titre et de volume	oui	oui	oui
Voir l'arbre des lieux	oui	oui	oui
Scanner / cataloguer un nouveau titre	non	oui	oui
Ajouter ou retirer un volume	non	oui	oui
Déplacer un volume entre lieux	non	oui	oui
Éditer les métadonnées d'un titre	non	oui	oui
Créer / éditer les emprunteurs	non	oui	oui
Enregistrer / retourner des prêts	non	oui	oui
Éditer l'arbre des lieux	non	oui	oui
Ouvrir <code>/admin</code>	non	non	oui
Créer / désactiver des utilisateurs	non	non	oui
Éditer les données de référence	non	non	oui
Restaurer depuis la corbeille	non	non	oui
Supprimer définitivement de la corbeille	non	non	oui
Éditer les paramètres système	non	non	oui

6.2 Sessions et timeout d'inactivité

Après la connexion, mybibli pose un cookie de session qui voyage avec chaque requête suivante. Le cookie est :

- `HttpOnly` — non exposé à JavaScript ;
- `SameSite=Lax` — envoyé sur les navigations same-site de haut niveau mais pas sur les requêtes cross-site ;
- `Secure` (uniquement si `MYIBLI_COOKIE_SECURE` est positionné à `true` — voir chapitre 1).

La session a un **timeout d'inactivité** (défaut 30 minutes, configurable depuis `/admin > System > Loans`). Chaque requête authentifiée réinitialise le compte à rebours ; si aucune requête n'arrive dans la fenêtre, la session est effacée côté serveur et la requête suivante redirige vers la page de connexion.

Un petit *toast de keep-alive* apparaît dans un coin de l'écran environ deux minutes avant l'expiration — cliquez-le pour prolonger la session sans quitter la page courante.

6.3 Garde du dernier admin actif

mybibli refuse de laisser l'installation sans aucun admin actif. La garde s'applique à deux actions :

- **Désactiver un utilisateur.** Si l'utilisateur est le seul admin actif, la requête renvoie un 409 conflict et la désactivation est refusée.
- **Supprimer définitivement un utilisateur depuis la corbeille.** Même règle.

L'auto-désactivation est également bloquée sans condition (vous ne pouvez pas désactiver le compte avec lequel vous êtes connecté).

Attention

La garde ne compte que les admins *actifs*. Si vous désactivez en masse des admins et que la garde se déclenche, défaites en réactivant quelqu'un depuis l'onglet corbeille. Il n'y a aucun chemin de récupération via l'assistant ou la base sans SQL manuel.

6.4 Réinitialisation de mot de passe

mybibli n'implémente pas l'envoi de courriels de réinitialisation de mot de passe — les déploiements familiaux / petite communauté tirent rarement profit de la plomberie email. Un admin peut réinitialiser le mot de passe d'un autre utilisateur en :

1. Ouvrant `/admin > Users`.
2. Choissant l'utilisateur, cliquant **Désactiver**.
3. Cliquant **Réactiver** depuis l'onglet corbeille.
4. Re-créant l'utilisateur avec un mot de passe frais.

Un utilisateur qui connaît son mot de passe actuel peut le changer depuis la page `/account`.

6.5 Sessions anonymes

Les visiteurs anonymes obtiennent aussi une ligne de session (utilisée pour la protection CSRF et la livraison des mises à jour HTMX en cours). Les sessions anonymes expirent après sept jours d'inactivité et sont collectées par une tâche d'arrière-plan quotidienne — elles n'affectent pas les utilisateurs connectés.

Chapitre 7

Sauvegarde et restauration

Une bibliothèque auto-hébergée n’a pas de filet de sécurité dans le cloud. Ce chapitre couvre quoi sauvegarder, à quelle fréquence, et comment restaurer.

7.1 Quoi sauvegarder

Trois artefacts forment ensemble une sauvegarde mybibli complète :

1. **La base MariaDB.** Contient chaque titre, volume, lieu, contributeur, prêt, utilisateur, journal d’audit et paramètre. La perdre, c’est repartir de zéro.
2. **Les images de couverture.** Téléchargées au catalogage et persistées dans le volume Docker `mybibli_covers` (ou dans le chemin hôte du bind mount si vous avez basculé selon section 1.3). Les perdre, le catalogue fonctionne toujours mais chaque page de titre affiche un placeholder générique tant que vous n’avez pas re-fetché les métadonnées titre par titre (issue #213 — les installs antérieures à 1.1.4 sans ce volume perdaient les couvertures à chaque mise à jour du conteneur).
3. **Le fichier `.env`.** Contient les mots de passe (base, clés d’API non encore migrées). Le perdre, un re-déploiement vous oblige à re-saisir chaque credential.

Les images Docker elles-mêmes ne sont *pas* critiques — vous pouvez toujours les re-tirer depuis Docker Hub.

7.2 Procédure de sauvegarde

Base de données — MariaDB embarquée

Depuis l’hôte qui exécute la pile compose :

```
docker compose exec -T db mariadb-dump \  
-u root -p"$MYSQL_ROOT_PASSWORD" \  
--single-transaction --routines --triggers \  
mybibli \  
> backups/mybibli-$(date +%Y%m%d-%H%M%S).sql
```

Le drapeau `--single-transaction` est important : il fait un snapshot cohérent sans verrouiller les tables pendant la durée du dump. `--routines --triggers` préserve les routines stockées (mybibli n’en utilise pas pour l’instant, mais le drapeau est peu coûteux et future-proof).

Compressez le dump si l'espace disque est une préoccupation :

```
gzip backups/mybibli-*.sql
```

Base de données — MariaDB externe

Lancez `mariadb-dump` directement contre le serveur externe. Les utilisateurs DSM Synology peuvent aussi planifier une tâche *Hyper Backup* qui cible le paquet MariaDB — elle produit une sauvegarde binaire qui se restaure plus vite qu'un dump SQL.

Couvertures et `.env`

Pour un répertoire de couvertures monté en bind-mount sur l'hôte, une simple archive `tar` suffit :

```
tar czf backups/mybibli-files-$(date +%Y%m%d).tar.gz \
.env "$COVERS_HOST_PATH"
```

Pour l'install par défaut (volume Docker nommé), sauvegardez le volume via Docker — le chemin hôte d'un volume nommé varie selon la version de Docker et n'est pas stable, donc n'essayez pas de le `tar` directement :

```
docker run --rm \
  -v mybibli_covers:/source:ro \
  -v "$(pwd)/backups:/backup" \
  alpine \
  tar czf "/backup/mybibli-covers-$(date +%Y%m%d).tar.gz" -C /source .
tar czf "backups/mybibli-env-$(date +%Y%m%d).tar.gz" .env
```

Le conteneur `alpine` jetable monte le volume nommé en lecture seule, le répertoire `backups/` hôte en écriture, et écrit l'archive `tar`. Même forme pour restaurer — voir la section Restauration plus bas.

7.3 Cadence recommandée

- **Quotidien** — dump de base automatisé. Même sur un catalogue rarement modifié, un dump journalier garantit que la pire perte de données est bornée à une journée.
- **Hebdomadaire** — `covers` et `.env`. Ils ne changent que quand vous cataloguez de nouveaux titres ou faites tourner des clés ; hebdomadaire est amplement suffisant.
- **Copie hors-site** — au moins mensuelle. Rsync du répertoire `backups/` vers un NAS distant, un disque USB que vous échangez ou un bucket de stockage cloud. Un incendie qui consume le serveur domestique ne doit pas aussi consumer vos sauvegardes.

Astuce

Une sauvegarde qui n'a jamais été restaurée n'est pas une sauvegarde. Une fois par trimestre, restaurez votre dernier dump dans un environnement Docker jetable et vérifiez que les compteurs de titres et de prêts correspondent à ce que rapporte `/admin > Health` en production.

7.4 Procédure de restauration

Base de données

Pour le setup avec base embarquée :

```
docker compose stop mybibli
docker compose exec -T db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" \
    -e "DROP DATABASE mybibli; CREATE DATABASE mybibli CHARACTER SET utf8mb4;"
zcat backups/mybibli-20260513-1200.sql.gz | \
    docker compose exec -T db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" mybibli
docker compose start mybibli
```

Pour MariaDB externe, les mêmes commandes client `mariadb` fonctionnent contre le serveur externe.

Couvertures et `.env`

Pour un install bind-mount hôte :

```
tar xzf backups/mybibli-files-20260513.tar.gz
```

Pour l'install par défaut (volume nommé), restaurez via un conteneur jetable symétrique à la sauvegarde :

```
docker compose stop mybibli
docker run --rm \
    -v mybibli_covers:/target \
    -v "$(pwd)/backups:/backup:ro" \
    alpine \
    sh -c "rm -rf /target/* && tar xzf /backup/mybibli-covers-20260513.tar.gz -C /target"
tar xzf backups/mybibli-env-20260513.tar.gz
docker compose start mybibli
```

Redémarrez la pile pour que le conteneur `mybibli` prenne en compte le `.env` restauré.

7.5 Migration vers un nouvel hôte

La procédure de sauvegarde est aussi la procédure de migration :

1. Sur l'ancien hôte : produire un dump de base et une archive tar frais.
2. Transférer les deux sur le nouvel hôte.
3. Sur le nouvel hôte : installer Docker, récupérer le fichier compose depuis la release correspondant à la version du dump, placer `.env` et `covers/` à côté, restaurer la base, démarrer la pile.
4. Vérifier en comptant les titres sur `/admin > Health` et en contrôlant quelques pages de titre au hasard.

Attention

Le dump est lié à la version de *schéma*. Si l'hôte source tournait en 1.1.0 et que le nouvel hôte tire 1.5.0, les migrations du nouvel hôte mettront à niveau la base restaurée — c'est OK. L'inverse (restaurer un dump 1.5.0 sur une installation 1.1.0) n'est *pas* supporté et

échouera au boot. Restaurez toujours sur un hôte exécutant la même version mybibli ou plus récente.

Chapitre 8

Mise à jour et migration

Les mises à jour de routine sont conçues pour être sûres. L'exception est le saut $1.0.x \rightarrow 1.1.0$, un breaking change unique documenté ci-dessous.

8.1 Mises à jour de routine ($1.y \rightarrow 1.z$)

1. Lisez les notes de version pour chaque version entre votre version courante et la cible.
2. Prenez une **sauvegarde de base** fraîche (chapitre 7).
3. Mettez à jour le tag d'image dans `docker-compose.yml` (ou re-tirez `:latest` si vous suivez ce tag).
4. Lancez `docker compose up -d`. Le nouveau conteneur boote, exécute les nouvelles migrations contre la base existante, et reprend le service.
5. Surveillez les logs pour les erreurs de migration : `docker compose logs mybibli | grep -i migrate`.

Les migrations sont append-only — les changements de schéma sont forward-compatibles. Une installation 1.4 mise à jour en 1.5 garde toutes les lignes de 1.4 et applique les nouvelles tables / colonnes sur place.

Astuce

Pour les installations homelab, le tag Docker `:latest` est un défaut raisonnable. Pour des montages production (bibliothèque communautaire, association), épinglez une version explicite pour qu'un pull non surveillé n'amène pas une release sur laquelle vous n'avez pas lu les notes.

8.2 Sauter des versions intermédiaires

Passer d'une installation bien plus ancienne (1.3.x par exemple) directement à la version courante est supporté et sûr pour la base. Le moteur de migrations applique toutes les migrations non encore appliquées dans l'ordre des timestamps au boot, quelle que soit la distance sautée. Les migrations de schéma sont purement additives — aucun `DROP COLUMN`, aucun `DROP TABLE` dans aucune release — et les quatre backfills de données existants (le seed CSRF token de 1.1.0, le nettoyage `manually_edited_fields` de 1.1.8, etc.) sont idempotents.

Ce qui reste valable sur un saut multi-versions :

- **Sauvegardez avant la mise à jour.** Il n’y a pas de rollback automatique. Une migration qui échoue en cours d’exécution laisse la base dans un état incohérent ; la voie de récupération est la sauvegarde pré-mise à jour (chapitre 7).
- **Les couvertures JPG pré-1.1.4 sont perdues.** Jusqu’à 1.1.4, le volume `mybibli_covers` n’était pas déclaré ; les couvertures écrites avant cette version étaient perdues à chaque upgrade de conteneur. Sauter directement vers 1.7 ne les récupère pas — re-fetch via l’action admin **Bulk re-fetch missing covers** (ajoutée en 1.2) ou le bouton **Re-fetch metadata** par titre.
- **Le premier boot peut prendre quelques minutes.** Un saut qui exécute 20+ migrations contre un gros catalogue est silencieux côté UX — attendez que `/health` réponde avant de vous inquiéter.

Le saut 1.0.x → 1.1.0 est l’unique exception qui nécessite la procédure dédiée ci-dessous.

8.3 Le saut 1.0.x → 1.1.0

Attention

Ceci est un breaking change unique. Les installations 1.0.x sont sujettes au bug documenté dans [#173](#) : les migrations seed créent les identifiants `admin/admin` et `librarian/librarian` à chaque installation neuve, y compris en production, et contournent silencieusement l’assistant de première mise en service.

Le chemin recommandé dépend de si votre installation 1.0.x détient des données réelles :

Si votre installation 1.0.x est vide (aucun titre catalogué)

Le chemin le plus simple : effacer la base et réinstaller en 1.1.0+.

1. Arrêter la pile : `docker compose down`.
2. Supprimer le volume de base : `docker volume rm mybibli_mybibli_db_data` (ou l’équivalent sur votre hôte — `docker volume ls` les liste).
3. Tirer l’image 1.1.0 : `docker compose pull`.
4. Mettre à jour le tag d’image dans `docker-compose.yml` si vous aviez épinglé une version antérieure.
5. Démarrer : `docker compose up -d`.
6. Ouvrir `/setup` et parcourir l’assistant.

Si votre installation 1.0.x détient des données réelles

1. Prenez une sauvegarde de base (chapitre 7).
2. Prenez une sauvegarde covers + `.env`.
3. Connectez-vous comme l’admin seedé et changez le mot de passe *avant* la mise à jour. La mise à jour 1.1.0 elle-même est sûre, mais la rotation ferme la dernière fenêtre pendant laquelle les identifiants par défaut pourraient fuiter.
4. Mettre à jour le tag d’image dans `docker-compose.yml` et lancer `docker compose up -d`.

5. Le boot 1.1.0 détecte l'admin existant et n'active *pas* l'assistant — l'admin avec mot de passe tourné reste l'admin actif.

La nouvelle variable MYBIBLI_SEED_DEV_USERS

À partir de 1.1.0, la migration seed est gardée par MYBIBLI_SEED_DEV_USERS. Le défaut est 0 (pas de seed). Ne la positionnez à 1 que pour des flux de dev local ou des tests automatisés — jamais en production. L'assistant de première mise en service devient le chemin canonique d'onboarding.

8.4 Lire les notes de version

Chaque release mybibli publie une page GitHub Release avec :

- un paragraphe résumant le changement ;
- l'impact migration (aucun / additif / breaking) ;
- un lien vers le diff contre le tag précédent ;
- les PDF de ce manuel (anglais + français) en pièce jointe une fois le build PDF de release câblé dans la CI.

Lisez les notes avant de mettre à jour. Les breaking changes portent une étiquette *Breaking* bien visible.

8.5 Nouveautés de la 1.7.9

La version 1.7.9 est un bundle « qualité couvertures + UX admin + 4 durcissements ». Une correction visible côté utilisateur (qualité des couvertures), une nouvelle surface admin, et quatre clean-ups de code-review. *Deux nouvelles lignes K/V dans la table settings (initialisées aux anciennes valeurs codées en dur), aucun breaking change* ; mise à jour de routine docker `compose pull && docker compose up -d`.

Visible côté utilisateur :

- **#333 piste 1 — Google Books : la chaîne privilège désormais la variante imageLinks la plus haute résolution disponible.** Le provider ne lisait que `thumbnail`, même quand la réponse Google Books portait aussi `small` / `medium` / `large` / `extraLarge`. Le redimensionnement Lanczos 400px aval dans `CoverService::process_and_save_bytes` partait alors d'une source dégradée. La nouvelle chaîne de repli (`extraLarge -> large -> medium -> small -> thumbnail -> smallThumbnail`) prend la meilleure variante disponible par titre : les titres existants gagnent une couverture plus nette après re-fetch, et le titre rare qui n'expose que `medium` et plus mais pas `thumbnail` reçoit enfin une couverture. Clôt la piste (1) de l'enquête sur le taux d'échec couverture de 50 % en production ; la piste 3 (upload manuel) a déjà été livrée en v1.7.6 #335 ; la piste 2 (BDGest fetch cover par ISBN) reste ouverte pour un cycle séparé.

Visible côté admin :

- **#334 — délais d'attente metadata-chain + provider-health dans /admin > Système.** Deux délais précédemment codés en dur (timeout par fournisseur dans la chaîne) ou disponibles uniquement par variable d'environnement (timeout de la sonde provider-health) sont maintenant configurables depuis le bloc « Fournisseurs de métadonnées » de l'onglet

Système. Bornés à 1..=60 secondes, persistés dans la table K/V `settings` (migration 20260526075659), rechargés à chaud via `Arc<RwLock<AppSettings>>` — la prochaine exécution de la chaîne / le prochain tour de ping prend la nouvelle valeur sans redémarrage. Les variables d’environnement historiques `MYBIBLI_METADATA_CHAIN_PROVIDER_TIMEOUT_SECS` et `MYBIBLI_PROVIDER_HEALTH_TIMEOUT_SECS` restent valables en bootstrap one-shot via `config::migrate_legacy` (même schéma que #308 v1.7.1 pour le niveau de log). Utile pour les déploiements à DNS+TLS lents (fibre rurale, VPN, satellite).

Durcissement (aucun changement visible) :

- **#46 — la purge des sessions anonymes survit aux crash-loops.** La purge quotidienne des lignes anonymes de plus de 7 jours dormait précédemment 24 h fixes après chaque démarrage. Sur une instance qui boucle en crash ou redéploie plus vite que 24 h, la purge ne se déclenchait jamais et la table `sessions` grossissait sans limite. Le délai du prochain run est désormais calculé depuis une ligne K/V persistée `last_anonymous_session_purge_at` (migration 20260526075700) : si 24 h se sont déjà écoulées depuis le run précédent, la purge se déclenche immédiatement (rattrapage) ; sinon elle dort le reste. Le contrat §Task 4.3 est préservé pour les installations fraîches.
- **#28 — trio observabilité.** (a) La trace de retry de `LoanService::register_loan` distingue désormais une défaillance business non-transitoire qui suit un retry transitoire (par ex. un soft-delete concurrent entre les tentatives qui fait basculer la seconde en `BadRequest`) — la log lisait précédemment « `warn!("retrying")` » suivi immédiatement de l’erreur business, donnant l’impression que le retry causait l’erreur. (b) Les 4 sites « fire-and-forget » catalog qui spawnt `fetch_metadata_chain` passent désormais par un nouveau wrapper `tasks::metadata_fetch::spawn_fetch` qui capture les panics dans la future interne et les loggue via `tracing::error!` au lieu de les laisser disparaître avec le `JoinHandle`. (c) Le rappel single-tenant dans `CLAUDE.md` est désormais explicite : `sqlx::Error::PoolTimedOut` n’est volontairement *pas* retried.
- **#24 — passe de durcissement `templates_audit`.** Cinq cas-limites traités : (a) `strip_html_comments` réécrite en machine à états — un `<!--` non terminé ne consomme plus le reste du fichier source (précédemment toute violation d’inline markup située après un commentaire tronqué était invisible pour l’audit). (b) La même fonction utilise désormais des tranches `&str` au lieu de caster chaque octet en `char`, ce qui préserve l’UTF-8 dans les snippets de rapport d’échec. (c) Nouvelle `strip_svg_inner_blocks` qui masque le contenu interne des SVG, de sorte qu’un `<svg><style>...</style></svg>` inline ne déclenche plus de faux positif sur la détection bare-`<style>`. (d) Les regex d’attributs de `<script>` et `<style>` utilisent désormais une grammaire consciente des guillemets `(?:[>"']|'["]*'|'[']*')*` de sorte qu’un `>` littéral à l’intérieur d’une valeur d’attribut entre guillemets ne termine plus le match prématurément. (e) Le filtre de bypass `src=/type=` est reconnu comme imprécis mais non exploitable ; pas de changement de code.
- **#196 — flake « course de saisie » du test `home-search`.** Le flake historique sur `tests/e2e/specs/journeys/home-search.spec.ts:224` (6 rétrospectives d’épopée d’affilée) est enfin clos. `simulateTyping` passe de `locator.pressSequentially` à `keyboard.type` après un `focus()` explicite : la première API re-focusait l’élément entre les actions de touche et pouvait perdre une frappe sous le parallélisme par défaut. Le poll d’état d’URL passe de 2 s à 5 s pour donner de la marge à la chaîne recherche debouncée + swap HTMX + `hx-push-url` sous charge.

8.6 Nouveautés de la 1.7.8

La version 1.7.8 est un patch « durcissement » apportant une correction UX visible et trois clean-ups de code-review. *Aucune migration de schéma, aucun breaking change*; mise à jour de routine `docker compose pull && docker compose up -d`.

Visible côté utilisateur :

- **#136 — les modales de suppression concurrente affichent un retour au lieu de disparaître en silence.** Avec deux bibliothécaires sur la même page `/borrower/:id`, `/contributor/:id` ou la modale de retour de prêt, le clic du second sur Supprimer disparaissait silencieusement : la handler renvoyait 404, le défaut `AppError::NotFound` reciblait sur `#feedback-list` et ces pages ne déclarent pas ce slot — le swap HTMX échouait sans message. Les trois handlers GET de déclenchement de modale renvoient désormais 200 + un fragment de feedback en ligne + `HX-Retarget` vers le slot existant de la page (`#borrower-feedback`, `#contributor-feedback` ou la résolution `?target=` pour le flux retour-prêt). Le second bibliothécaire voit un toast localisé « déjà supprimé par une autre session » dans les 4 langues supportées.

Durcissement (aucun changement visible) :

- **#39 — bascule de session de login atomique.** `services::auth::authenticate_session` exécutait l'INSERT de la nouvelle session et le UPDATE de l'ancienne session anonyme comme deux requêtes indépendantes. Une défaillance partielle entre les deux laissait une ligne anonyme orpheline à côté de la nouvelle ligne authentifiée (les deux portaient des jetons CSRF valides pour le même navigateur). Les deux requêtes tournent maintenant dans une seule `sqlx::Transaction` — bascule atomique ou rollback complet.
- **#47 — combler 4 trous de couverture du suite de tests CSRF.** Quatre tests d'intégration Rust à forte valeur ajoutés : branche HTMX du POST language, logout via le jeton en champ de formulaire (la forme utilisée par la nav-bar), rejouement d'un jeton périmé contre une session rotée, et une assertion sur le corps de la 403 prouvant que l'enveloppe de rejet porte bien le markup `FeedbackEntry`. Les 5 autres éléments E2E (parallèle-pollution + sélecteurs) sont reportés à une passe E2E ultérieure.
- **#217 — resserrement du scope de confirmation modal.** `modal.js::originatesFromConfirm` retournait vrai pour N'IMPORTE QUEL `<form>` descendant de la dialog ouverte. Une future modale avec un formulaire imbriqué aurait étiqueté ses soumissions avec `X-Modal-Confirm: true` et déclenché à tort le `ModalConfirmRetargetGuard` côté serveur. Le prédicat correspond désormais seulement au premier formulaire de la dialog (`dialog.querySelector("form")`) — qui est le formulaire Confirm par construction du macro. Une E2E regression-guard injecte un second formulaire dans une dialog ouverte et assert que l'en-tête n'est PAS envoyé.

8.7 Nouveautés de la 1.7.7

La version 1.7.7 est un patch « rigueur de test » qui apporte une amélioration visible et sept clean-ups de code-review fermant des patterns E2E silencieux et verrouillant des contrats de concurrence et de résolution de locale. *Aucune migration de schéma, aucun breaking change*; mise à jour de routine `docker compose pull && docker compose up -d`.

Visible côté utilisateur :

- **#336 — vue série en cartes.** La page `/series/:id` listait chaque position comme une

petite case colorée ne portant que le numéro de volume. Désormais chaque position remplie est rendue comme une carte avec la couverture en haut (ou l'icône de type média par défaut si aucune couverture n'a encore été téléchargée), un préfixe « Vol. N » en couleur accent, le titre (tronqué) et le contributeur principal. Les positions manquantes sont rendues comme des cartes en pointillés portant aussi « Vol. N » pour que les collectionneurs repèrent d'un coup d'œil les volumes manquants. La grille est responsive (2 colonnes sur mobile jusqu'à 6 colonnes sur écran très large) et préserve le contrat ARIA `role="grid" / role="gridcell"` pour que la navigation au lecteur d'écran reste identique.

Rigueur de test (aucun changement visible) :

- **#43 — helper DB-snapshot sur le rejet POST contributeur.** Un nouveau helper Playwright `captureEntityCounts()` prouve qu'un rejet 403/303 d'un POST anonyme a bien laissé la DB intacte (l'assertion de status code seule pourrait laisser passer une future régression où le middleware rejette avec le bon status tout en laissant la mutation s'exécuter).
- **#62 — vraies assertions dans le spec admin permanent-delete.** Le spec précédent enrobait chaque assertion dans `if (tableExists) { ...}`, ne testant rien silencieusement sur une DB CI vierge. La réécriture seed des entités soft-deleted par-test et exerce le flux « tape pour confirmer » de bout en bout.
- **#88, #89 — concurrence + résolution de locale dans System.** Deux branches de test manquantes autour du design optimistic-locking partitionné (deux lignes de réglages distinctes ne se collisionnent pas) et de la chaîne de résolution de locale (Accept-Language et préférence utilisateur authentifiée gagnent toutes deux sur le default global).
- **#96 — race du premier-lancement concurrent (test d'intégration).** Deux POSTs Step 1 lancés par `tokio::spawn` racent le garde « premier admin » et assertent qu'il n'existe exactement qu'une ligne admin après — l'invariant AC13 qui était unit-testé par forme mais jamais exercé bout-à-bout.
- **#119 — le spec dashboard seed son propre volume non-rangé.** Le smoke « What needs attention » ressortait silencieusement sur une DB vierge. L'indicateur unshelved est désormais correctement seedé via `scanTitleAndVolume()` ; l'indicateur overdue nécessite de pré-dater un prêt et est suivi séparément.
- **#120 — contrat single-active-filter.** Deux nouveaux tests de rendu verrouillent le contrat selon lequel `?filter=overdue&q=X&sort=Y` court-circuite la branche search-fragment (la liste-indicateur possède la réponse ; `#browse-results` reste dans le DOM comme cible HTMX mais ne porte aucun contenu de recherche).

8.8 Nouveautés de la 1.7.6

La version 1.7.6 est un patch ciblé : deux corrections de fonctionnalités visibles utilisateur et trois durcissements issus du backlog de revue de code. *Pas de migration de schéma, pas de breaking change* ; `docker compose pull && docker compose up -d` suffit.

Visible utilisateur :

- **#331 — type de média modifiable sur /title/:id.** Une BD ou un manga catalogué via son ISBN restait verrouillé en « livre » à vie — le formulaire d'édition affichait `media_type` en lecture seule. Le formulaire propose maintenant un menu déroulant avec les six valeurs (`book`, `bd`, `cd`, `dvd`, `magazine`, `report`) ; tout changement déclenche une entrée `admin_audit`

(`title_media_type_edit`) avec le couple avant/après. Pratique pour corriger une BD que la BnF a résolue comme livre au moment du scan.

- **#335 — téléversement manuel de couverture.** Le fallback de la chaîne (Open Library Covers API) rate la plupart des BDs francophones, des mangas et des livres techniques de petits éditeurs : son index est centré sur le marché anglophone. Cliquez *Téléverser une couverture* sous la vignette sur `/title/:id` (rôle Bibliothécaire+) pour déposer un JPG / PNG / WebP / GIF (10 Mio). L'image passe par exactement la même chaîne de décodage / redimensionnement / encodage JPEG que les couvertures téléchargées par les providers ; le fichier sur disque reste compatible. `cover_image_url` est ajouté à `manually_edited_fields`, ce qui exclut le titre de la prochaine action « Récupérer les couvertures manquantes » — votre couverture choisie est conservée.

Durcissement (sans changement visible) :

- **#109 — couverture de test des cartes « ajouts récents ».** Les tests de rendu de la page d'accueil laissaient `publication_date` et `cover_image_url` à `None`, n'exerçant jamais les branches `{% if let Some(d) = ...%}`. Un mauvais format de date ou une balise oubliée serait passé inaperçu en unitaire. Nouvelle factory `fake_search_result_full` et nouveau test de rendu.
- **#40 — normalisation des routes exemptées CSRF.** La comparaison contre la liste exemptée était littérale ; `POST /login/` ou `POST //login` (variantes que le routeur Axum dispatche identiquement vers `POST /login`) passaient par la validation CSRF au lieu d'être contournées. Nouvelle fonction `normalize_exempt_path` qui retire le slash final et fusionne les slashes répétés — les formes percent-encodées ne sont volontairement *pas* décodées (la tentative de bypass `/login%2F..%2Fadmin` reste rejetée).
- **#36 — bypass du résolveur de session sur les routes statiques.** `session_resolve_middleware` traversait le résolveur sur *chaque* requête, y compris `/static/*`, `/covers/*` et `/logo/*`. Un seul chargement de la page d'accueil déclenchait des dizaines de requêtes d'assets, chacune lisant une ligne `sessions` (et insérant une ligne anonyme sur un onglet vierge). Le middleware court-circuite désormais ces préfixes en plus du bypass `/health` existant.

8.9 Nouveautés de la 1.7.5

La version 1.7.5 est un patch ciblé qui groupe un correctif visible utilisateur et sept commits de durcissement issus de code-review-findings. *Aucune migration de schéma, aucune cassure* ; `docker compose pull && docker compose up -d` habituel.

Visible utilisateur :

- **#325 — ajout / retrait de contributeur silencieusement cassé sur `/title/:id`.** Cliquer sur *+ Ajouter un contributeur* sur la page de détail d'un titre ouvrait bien le formulaire et acceptait le nom et le rôle, mais *Enregistrer* ne produisait aucun effet visible. Cause racine : le formulaire pointe `hx-target="#feedback-list"`, mais le slot était déclaré uniquement sur `/catalog` et `/admin` — pas sur `/title/:id`. Le même défaut affectait silencieusement le bouton *Retirer*. Régression du fix v1.7.2 qui avait ajouté le flux de détail sans porter le slot. Un garde-fou d'audit templates panique désormais si une page référence `hx-target="#feedback-list"` sans déclarer `id="feedback-list"`.

Durcissement (aucune cassure utilisateur) :

- **#91 — re-rendu du formulaire sur erreur de validation.** Admin → Système → Prêts/Langue retournent 400 avec le formulaire re-rendu (la valeur saisie est préservée) et un retour d'erreur inline, au lieu d'un 400 vide silencieusement avalé par HTMX 2.0.
- **#42 — jeton CSRF en premier-enfant strict.** L'audit `forms_include_csrf_token` exige désormais `_csrf_token` comme tout premier `<input>` de chaque formulaire POST, pas seulement parmi les cinq premiers.
- **#48 — audit des littéraux HTML Rust.** Nouveau test frère `rust_emitted_post_forms_include_csrf` parcourt `src/**/*.rs` et refuse tout `<form method="POST">` en chaîne littérale qui ne déclare pas son jeton CSRF à proximité. Verrouille le site existant `src/routes/locations.rs:301` ; tout nouveau formulaire émis depuis Rust devra déclarer son jeton.
- **#220 — HX-Trigger en forme JSON.** `modal_confirm_retarget_guard` tolère désormais la forme JSON-objet du header HX-Trigger en plus de la forme virgule-séparée. Défensif : l'émetteur actuel utilise la forme virgule.
- **#31 — scanner-guard respect du pipeline d'entrée.** Le JavaScript `scanner-guard` qui transmet les rafales de scanner USB vers le champ texte d'un modal ouvert respecte désormais `maxLength` et passe outre pendant une composition IME active. Les sous-items 3/4/5 étaient déjà classés N/A à la forme actuelle des modaux.
- **#90 — DRY admin_system.** La requête UPDATE de verrou optimiste dupliquée entre `save_setting` et `run_provider_update` est consolidée ; le second délègue désormais et `map_errs` le message de conflit pour interpoler le label du provider.
- **#152 — test /series anonyme.** Ajoute le test de garde de rôle manquant qui verrouille le contrat FR95 d'accès anonyme à `/series` (miroir de `/catalog` et `/locations`).

8.10 Rollback

Le rollback *n'est pas* un flux de première classe. Les migrations de schéma avancent uniquement — il n'y a pas de `down.sql`. La stratégie de récupération supportée est :

1. Restaurer la sauvegarde de base pré-mise-à-jour (chapitre 7).
2. Réépingler le tag d'image à la version antérieure.
3. Démarrer la pile.

Cela fonctionne car la sauvegarde est cohérente avec le schéma antérieur. Tenter de pointer une image plus ancienne sur une base fraîchement migrée échoue au boot.

Chapitre 9

Bonnes pratiques

Leçons concentrées tirées de déploiements réels. Chaque section est optionnelle — prenez ce qui colle à votre échelle.

9.1 Hiérarchie de stockage

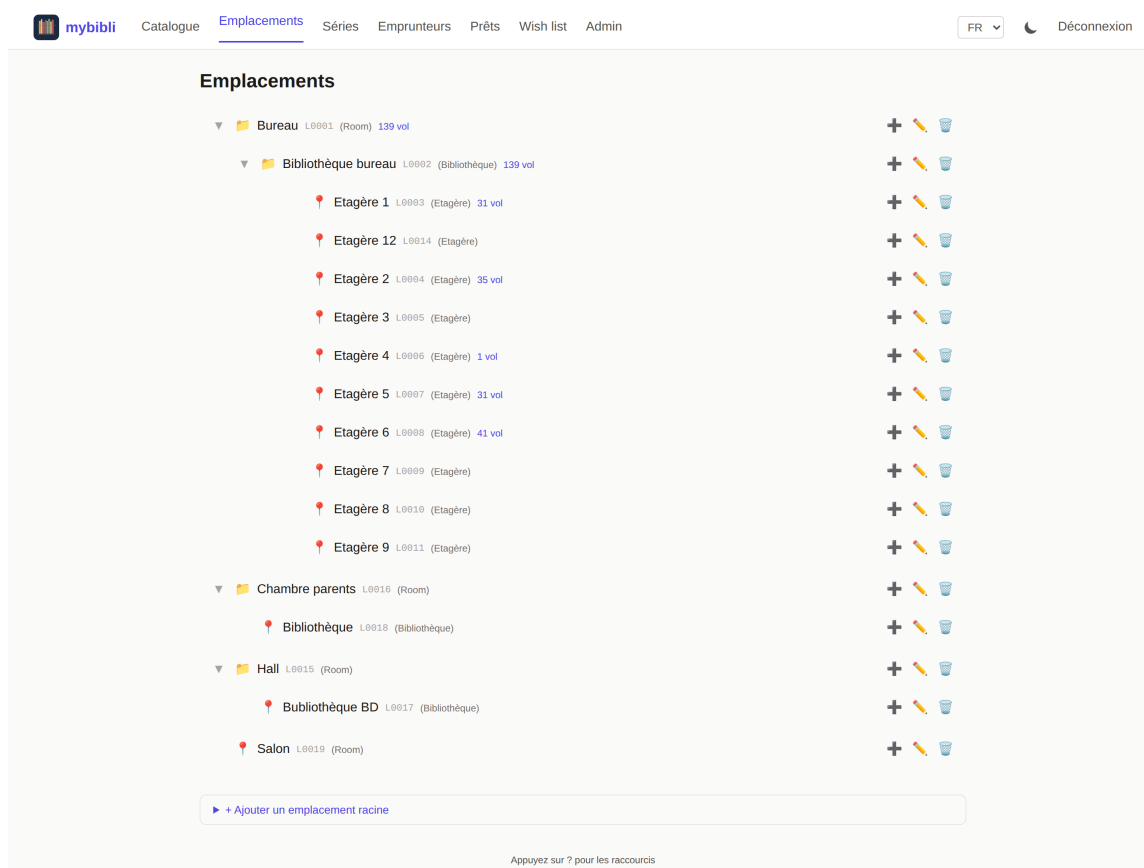


Fig. 9.1 : La page *Emplacements* sur une installation réelle : une hiérarchie domestique avec un bureau, une bibliothèque dans la pièce, dix étagères et deux pièces de plain-pied — chaque nœud porte son code L et un compteur de volumes à jour.

- **Trois niveaux suffisent en général.** Pièce → étagère → rangée couvre la plupart des foyers. Un quatrième niveau (par exemple « boîte à l'intérieur de la rangée ») devient de

la paperasse à maintenir.

- **Nommez les lieux comme vous en parlez.** « Étagère du salon, en haut » bat « SAL-E01-R01 ». Les noms apparaissent sur les étiquettes ; traitez-les comme du contenu visible.
- **Imprimez les codes L une seule fois.** Réimprimer parce que vous avez rééquilibré les étagères est du travail perdu — les codes L sont des identifiants stables ; le nom humain sur l'étiquette peut dériver et c'est OK.

9.2 Planches de codes V

- Imprimez les codes V en planches de 50 à 100. Des autocollants pré-imprimés dans un porte-étiquettes près du poste de scan rendent le catalogage *rapide*.
- **Collez le code V avant de scanner.** Cela élimine un futur mode de panne où vous mal-étiquetez un livre et devez le re-trouver par scan.
- **Placez l'étiquette toujours au même endroit.** Intérieur de la 4^e de couverture, coin supérieur droit, parallèle au dos — choisissez une règle et tenez-vous-y. Le futur-vous lors d'un audit remerciera le présent-vous.

9.3 Genre versus Dewey

mybibli supporte à la fois des genres larges et des codes Dewey. Choisissez l'un des deux comme aide principale à la navigation :

- **Utilisez les genres** quand le public navigue par humeur (« je veux un thriller ») et que la collection est assez petite pour que « Thriller » soit suffisamment granulaire.
- **Utilisez Dewey** quand le public navigue par sujet (« livres sur l'histoire de France du XVIII^e siècle ») ou quand la collection a atteint la taille où « Documentaire » n'est plus un filtre utile.
- Utiliser les deux est OK ; mybibli ne force pas le choix.

9.4 Cadence d'audit

- **Trimestriel** pour les collections sous quelques centaines de volumes.
- **Mensuel** pour les collections au-dessus, spécialement si plus d'une personne catalogue.
- Lancez toujours un audit *après* un grand rééquilibrage de rayons — déplacer des livres physiquement sans le dire à mybibli est la première source de bugs « où est ce volume ? ».

9.5 Comptes utilisateur

- **Un humain, un compte.** Partager un seul compte admin entre deux personnes fait perdre la piste d'audit et rend la garde du dernier admin moins utile.
- **La plupart des éditeurs doivent être Bibliothécaires, pas Admins.** Le panneau admin gère des préoccupations d'installation et de nettoyage rare ; vous n'avez pas besoin de droits admin pour cataloguer ou prêter.

- **Changez le mot de passe admin seedé au premier login.** Sur 1.1.0+ l'assistant le force ; sur les installations 1.0.x existantes, faites-le manuellement avant d'exposer l'instance sur un réseau.

9.6 Prêts

- **Gardez des emprunteurs identifiables.** « Marie L. » bat « Marie » dès la seconde Marie qui pointe son nez. Email ou téléphone dans le champ notes aide pour les relances.
- **Parcourez la liste des retards une fois par semaine.** L'indicateur du dashboard vous houspille pour une raison — les prêts en retard cumulés sur un an deviennent des livres que vous ne possédez plus.
- **Notez les dégâts au retour.** Basculer un volume retourné en *Usé* ou *Endommagé* capture l'information pendant que l'emprunteur est devant vous ; chasser le coût plus tard de mémoire fonctionne rarement.

9.7 Fournisseurs de métadonnées

- **Limitez les clés.** Vous n'avez pas besoin des trois fournisseurs à clé dès le premier jour — BnF + Open Library couvrent les livres gratuitement. Ajoutez OMDb + TMDb le jour où vous cataloguez votre premier DVD.
- **Traitez les clés comme des secrets.** `.env` est dans `.gitignore` ; conservez les sauvegardes de `.env` chiffrées (chapitre 7).

9.8 Mises à jour

- **Lisez les notes de version avant de tirer `:latest`.** Voir chapitre 8.
- **Sauvegardez juste avant chaque mise à jour.** Même les migrations « additives » ont une surface de panne non-nulle ; une sauvegarde minute-avant transforme une inquiétude en formalité.
- **Épinglez une version pour la production.** Les auto-updates conviennent à une instance familiale ; pour une bibliothèque communautaire, un saut majeur non surveillé peut interrompre le service.

Chapitre 10

Dépannage

Quand quelque chose tourne mal, commencez par les logs et le tableau de bord `/admin > Health`. Ce chapitre catalogue les problèmes que les utilisateurs rencontrent vraiment, par ordre de fréquence approximatif.

10.1 Lire les logs

```
docker compose logs --since 10m mybibli
docker compose logs -f mybibli
```

Les logs sortent en JSON structuré. Les champs les plus utiles sont `level`, `target` (quel module a logué cette ligne), `message`, et toute paire clé=valeur contextuelle que la ligne porte (`user_id`, `volume_id`, ...).

Pour filtrer :

```
docker compose logs --since 10m mybibli | grep -i error
```

10.2 L'application ne démarre pas

Symptôme : le conteneur s'arrête immédiatement

Lancez `docker compose logs mybibli` et cherchez l'une des lignes :

- `missing required environment variable: DATABASE_URL` — le fichier `.env` est absent ou la variable n'est pas positionnée. Voir section [1.5](#).
- `Failed to create database pool` — l'hôte / port / identifiants dans `DATABASE_URL` sont faux, ou la base n'est pas joignable. Depuis l'hôte, essayez `docker compose exec mybibli sh -c 'apk add curl; curl <db-host>:<db-port>'` pour confirmer la joignabilité.
- `Failed to run database migrations` — généralement un souci de permission. Confirmez que l'utilisateur DB a `ALL PRIVILEGES` sur la base (voir section [1.4](#)).
- `Address already in use` — le port hôte publié est pris. Changez `HOST_PORT` dans `.env`.

Symptôme : chaque page renvoie 404

Le conteneur est levé mais les routes manquent. Cause la plus fréquente : vous avez tiré un tag d'image qui n'existe pas (faute de frappe) et Docker est retombé sur un vieux tag obsolète. `docker compose pull` puis redémarrez.

10.3 Impossible de se connecter

Symptôme : « Invalid credentials » sur une installation 1.0.x neuve

Les identifiants seedés sont `admin/admin`. S'ils ne marchent pas non plus :

- la migration de base qui les crée a échoué silencieusement — cherchez les erreurs de migration dans les logs ;
- l'admin précédent a déjà tourné le mot de passe (vérifiez avec qui a installé mybibli) ;
- vous regardez une installation 1.1.0+, où les identifiants seedés sont bien plantés par la migration mais immédiatement *soft-deleted* par la garde issue-#173 au démarrage — utilisez plutôt l'assistant à `/setup`.

10.4 Utilisateurs résiduels dans la table users après l'assistant

Symptôme : quatre utilisateurs en base, un seul créé dans l'assistant

Si vous inspectez la table `users` directement (via phpMyAdmin ou le CLI MariaDB) juste après avoir terminé `/setup`, vous verrez *quatre* lignes alors que vous n'avez saisi qu'un seul nom d'utilisateur et un seul mot de passe. C'est intentionnel. Trois de ces lignes ne peuvent pas servir à se connecter.

username	role	active	deleted_at
admin	admin	1	rempli, $\approx$ heure du premier boot
librarian	librarian	1	rempli, $\approx$ heure du premier boot
SYSTEM	system	0	NULL
<votre-username>	admin	1	NULL

- `admin` et `librarian` sont plantés par les migrations de seed (qui tournent à chaque installation neuve pour le flux dev / E2E) puis immédiatement *soft-deleted* par la garde issue-#173 au démarrage. Leur `deleted_at` est rempli, donc la requête de login (qui filtre `deleted_at IS NULL`) les refuse.
- `SYSTEM` est un compte utilitaire non connectable, utilisé pour attribuer les entrées du journal d'audit produites par la tâche de fond de purge automatique à 30 jours. Son rôle est `system` (hors de l'énum de login `admin/librarian`), son `password_hash` est la chaîne littérale `NO_LOGIN_SYSTEM_USER_HASH_NOT_A_VALID_ARGON2_STRING`, et son flag `active` vaut 0.

Astuce

Pour vérifier que les lignes *soft-deleted* sont inertes, essayez de vous connecter avec `admin/admin` : la tentative est refusée. Le panneau `/admin > Utilisateurs` filtre la ligne `SYSTEM` ainsi que toute ligne avec `deleted_at` rempli, donc l'interface admin n'affiche que le compte que vous avez créé dans l'assistant.

Les deux lignes `soft-deleted` sont physiquement supprimées de la table par le planificateur de purge automatique à 30 jours — attendez-vous à les voir disparaître environ un mois après la date de boot inscrite dans `deleted_at`.

Symptôme : la connexion redirige vers `/login`

Le cookie de session est rejeté par le navigateur. Causes :

- `MYBIBLI_COOKIE_SECURE=true` sur un déploiement plain-HTTP — le navigateur refuse d'accepter un cookie Secure en HTTP. Positionnez la variable à `false` ou frontez l'instance en HTTPS.
- Cookies bloqués par politique du navigateur (fenêtre privée avec blocage strict des cookies tiers, etc.). Testez dans un profil par défaut.

Symptôme : verrouillé dehors — dernier admin désactivé

La garde du dernier admin actif devrait empêcher cela, mais si un admin précédent a été désactivé puis supprimé définitivement, la récupération demande un accès SQL direct :

```
docker compose exec db mariadb -u root -p"$MYSQL_ROOT_PASSWORD" mybibli \  
-e "UPDATE users SET deleted_at=NULL, active=1 WHERE username='votre-admin';"
```

Ensuite connectez-vous normalement.

10.5 Les métadonnées ne se résolvent jamais

Symptôme : le titre squelette reste non résolu

Ouvrez `/admin > Health`. Regardez l'état de chaque fournisseur. Une croix rouge veut généralement dire :

- le fournisseur est vraiment down (rare pour BnF / Open Library ; plus fréquent pour les services plus petits) ;
- la clé d'API est absente ou invalide — ouvrez `/admin > System > Metadata Providers` et re-saisissez la clé ;
- l'hôte ne peut pas joindre l'upstream. Confirmez avec `docker compose exec mybibli sh -c "wget -q https://www.googleapis.com/books/v1 -O -"` ou équivalent.

Symptôme : seuls les ISBN français se résolvent

Vous avez désactivé (ou jamais positionné) les clés Google Books / Open Library. BnF est excellent pour les titres édités en France mais limité pour les titres en langues étrangères. Ajoutez au moins Open Library (sans clé requise).

10.6 La recherche renvoie trop peu de résultats

Symptôme : des titres connus ne remontent pas

- Confirmez que le titre n'est pas `soft-deleted` (vérifiez `/admin > Trash`). La recherche cache les éléments `soft-deleted`.

- Confirmez que les contributeurs du titre sont orthographiés comme vous avez cherché. La recherche mybibli est floue mais pas magique ; « Camus » trouvera « Albert Camus » mais « Camu » peut ne rien donner.

Symptôme : la recherche est lente

Pour les collections au-delà de 10 000 titres, les requêtes qui combinent contributeur et texte libre peuvent prendre une seconde ou deux. C'est une limitation connue ; des index prévus combleront l'écart.

10.7 Images de couverture manquantes

Symptôme : chaque titre affiche le placeholder

Le répertoire des couvertures est vide ou illisible. Confirmez :

- que `docker-compose.yml` déclare le volume nommé `mybibli_covers` (ou un bind mount sur `COVERS_HOST_PATH`) mappé sur `/app/covers`. Les installs antérieures à 1.1.4 ne déclaraient PAS ce volume — voir la sous-section suivante ;
- que la récupération de métadonnées a effectivement renvoyé une URL de couverture (regardez la page du titre — si l'URL manque, le fournisseur ne l'a pas fournie).

Symptôme : les couvertures ont disparu après mise à jour de l'image

Vous avez mis à jour depuis une install antérieure à 1.1.4 et toutes les couvertures précédemment cataloguées ont disparu. Il s'agit de l'issue #213 — l'ancien modèle compose ne persistait pas `/app/covers`, donc le layer en écriture portant ces JPG a été détruit quand le nouveau conteneur a remplacé l'ancien.

Correctif : récupérez le `docker-compose.yml` de la 1.1.4, puis lancez `docker compose down && docker compose up -d`. Les couvertures suivantes seront persistées dans le volume `mybibli_covers`.

Récupération des couvertures perdues : ouvrez la page de détail de chaque titre concerné et cliquez sur *Re-télécharger les métadonnées* pour repeupler depuis la chaîne de fournisseurs. Une action en masse est suivie dans l'issue #214.

10.8 Où signaler un bug

Problèmes que vous ne pouvez pas résoudre, ou comportement qui contredit ce manuel : ouvrez un ticket sur <https://github.com/guycorbaz/mybibli/issues>. Le template d'issue demande la version (visible en pied de page), la méthode d'installation et les étapes pour reproduire — merci de les renseigner.

Chapitre 11

API & intégrations

mybibli expose une petite API JSON HTTP sous `/api/v1/*` qui permet à des scripts externes et à des outils d’intelligence artificielle de lire votre catalogue et de modifier la classification des titres. Cette API est servie par le même serveur web que l’interface humaine — pas de démon séparé, pas de port supplémentaire à ouvrir.

Ce chapitre couvre :

- la création d’une clé d’API dans le panneau d’administration,
- les deux portées (*lecture seule* et *lecture + écriture*),
- les points d’entrée exposés sous `/api/v1/*`,
- trois exemples de bout en bout (curl, Python, boucle de classificateur IA).

11.1 Pourquoi une API HTTP ?

Le cas d’usage quotidien est de déléguer le travail de classification à un outil externe — par exemple demander à un LLM de suggérer un code Dewey, un genre ou un sous-titre propre pour chaque nouveau livre en anglais. Le LLM parcourt le catalogue avec `GET /api/v1/titles`, récupère le détail de chaque entrée via `GET /api/v1/titles/{id}`, calcule une suggestion, puis l’écrit avec `PATCH /api/v1/titles/{id}`. Chaque `PATCH` est journalisé pour pouvoir revenir en arrière si un modèle a mal travaillé.

L’API n’est *pas* prévue pour l’import en masse à haut débit — pour cela, utilisez directement la base SQL lors d’une fenêtre de maintenance. Elle *est* prévue pour le genre d’écritures lentes, réfléchies et opinionées qui s’insèrent dans une boucle d’agent LLM.

11.2 Créer une clé d’API

Les clés sont gérées dans **Administration** → **Clés d’API**. Seuls les utilisateurs Admin voient cet onglet.

1. Cliquez sur **Administration** dans la barre de navigation.
2. Ouvrez l’onglet **Clés d’API**.
3. Saisissez un **libellé** — un nom que vous seul voyez, par ex. « *Classificateur IA* » ou « *Script de sauvegarde* ». Le libellé sert au journal d’audit ; la clé elle-même est aléatoire et

indépendante du libellé.

4. Choisissez une **portée** :

- **Lecture seule** — uniquement les points d'entrée GET. Idéal pour des tableaux de bord en lecture, des outils de miroir ou des scripts de curiosité.
- **Lecture + écriture** — autorise en plus PATCH `/api/v1/titles/{id}` pour une liste blanche restreinte de champs (genre, Dewey, description, sous-titre). À utiliser pour des agents LLM ou des outils de curation.

5. Cliquez sur **Créer la clé**. Une boîte de dialogue affiche la valeur complète une seule fois — copiez-la maintenant. Une fois fermée, il n'y a aucun moyen de la récupérer ; si vous perdez la clé, révoquez-la et créez-en une nouvelle.

La clé a la forme `mybibli_<portée>_<aléa40>`, où `<portée>` vaut `ro` ou `rw`. Seuls le hachage `argon2` et les 12 premiers caractères du texte clair (le préfixe) sont persistés en base ; le préfixe sert à restreindre les candidats lors de l'authentification. Le hachage ne peut pas être inversé.

11.3 S'authentifier

Chaque requête `/api/v1/*` doit transporter l'un des deux en-têtes :

- `Authorization: Bearer <clé>` — préféré.
- `X-API-Key: <clé>` — solution de repli pratique.

Une clé absente ou invalide retourne 401 `Unauthorized` avec une enveloppe JSON :

```
{ "error": "unauthorized",
  "reason": "missing_api_key" }
```

Une clé en lecture seule sur un point d'entrée d'écriture retourne 403 `Forbidden` avec la portée manquante explicitement nommée :

```
{ "error": "forbidden",
  "reason": "insufficient_scope",
  "scope_required": "write" }
```

La protection CSRF est volontairement court-circuitée pour `/api/*` parce que l'authentification par jeton ne repose pas sur les cookies, et le risque de jeu cross-site contre lequel CSRF protège ne s'applique pas.

11.4 Points d'entrée

11.4.1 GET `/api/v1/titles`

Liste paginée. Accepte `?page=N`, `?q=<recherche>`, `?genre_id=N` et `?dewey_prefix=8`.

Retour :

```
{ "items": [ { "id": 42, "title": "...",
               "subtitle": null,
               "language": "fr",
               "media_type": "book",
               "isbn": "9782...",
```

```

        "publisher": "...",
        "publication_year": 2020,
        "genre_id": 7,
        "genre_name": "Roman",
        "dewey_code": "843.92" },
    ... ],
    "page": 1, "total_pages": 12,
    "total_items": 287 }

```

11.4.2 GET /api/v1/titles/{id}

Détail complet, incluant les contributeurs, les volumes (avec leur emplacement) et les affectations de série. Le champ `version` est le verrou optimiste — conservez-le à côté des données que votre script va modifier.

11.4.3 GET /api/v1/genres, /locations, /series

Données de référence. À utiliser quand un LLM a besoin de connaître les genres existants avant de suggérer un `genre_id`.

11.4.4 GET /api/v1/dewey/{prefix}

Recherche par bucket. Passez un préfixe Dewey partiel (par ex. 8 pour la littérature) et récupérez tous les titres dont le `dewey_code` commence par ce préfixe. Utile pour « trouver tous les livres qui ont déjà un code Dewey en 800 que je puisse comparer ».

11.4.5 PATCH /api/v1/titles/{id}

Portée écriture uniquement. Modifie les quatre champs en liste blanche :

- `subtitle` — peut être null.
- `description` — peut être null.
- `dewey_code` — peut être null ; chiffres, points et slashes uniquement.
- `genre_id` — doit référencer un genre existant et non supprimé.

Le corps JSON suit une sémantique de mise à jour partielle :

- Champ **omis** → pas de changement.
- Champ à `null` → effacement vers NULL (pour les trois champs nullable ; `null` sur `genre_id` retourne 400).
- Champ avec une valeur → mise à jour.

`version` est obligatoire. En cas de désaccord la réponse est 409 **Conflict** et contient la version attendue et la version reçue — relisez la donnée et réessayez.

Un PATCH réussi écrit une ligne dans `admin_audit` avec l'action `api_patch_title`, l'attribution via l'admin émetteur (`user_id`) et la clé d'API (`details.via_api_key_id + details.via_api_key_label`), et un diff `old/new` par champ. Un PATCH sans changement effectif renvoie 200 mais NE bump PAS `version` et N'ÉCRIT PAS de ligne d'audit.

11.5 Exemples

11.5.1 curl

```
KEY="mybibli_ro_..."

# Lister la première page des titres.
curl -H "Authorization: Bearer $KEY" \
      https://library.example.org/api/v1/titles

# Afficher un titre en détail.
curl -H "Authorization: Bearer $KEY" \
      https://library.example.org/api/v1/titles/42

# PATCH un code Dewey (clé write requise).
KEY_WRITE="mybibli_rw_..."
curl -X PATCH \
      -H "Authorization: Bearer $KEY_WRITE" \
      -H "Content-Type: application/json" \
      -d '{"version": 3, "dewey_code": "813.54"}' \
      https://library.example.org/api/v1/titles/42
```

11.5.2 Python

```
import os, requests

BASE = "https://library.example.org/api/v1"
KEY  = os.environ["MYBIBLI_API_KEY"]
H    = {"Authorization": f"Bearer {KEY}"}

# Parcourir toutes les pages.
page = 1
while True:
    r = requests.get(f"{BASE}/titles?page={page}",
                     headers=H, timeout=30)
    r.raise_for_status()
    data = r.json()
    for t in data["items"]:
        print(t["id"], t["title"], t.get("dewey_code"))
    if page >= data["total_pages"]:
        break
    page += 1
```

11.5.3 Une boucle de classificateur LLM

Le flux d'écriture prévu ressemble à ceci — en pseudo-code :

```
for t in walk_titles(KEY_READ):
    if t["dewey_code"]:
        continue # déjà classifié
    detail = get(f"/titles/{t['id']}")
    suggestion = ask_llm(detail) # renvoie {"dewey_code": ...}
    if not suggestion.get("dewey_code"):
```

```
        continue
    patch(
        f"/titles/{t['id']}",
        json = {
            "version":    detail["version"],
            "dewey_code": suggestion["dewey_code"],
        },
        key   = KEY_WRITE,
    )
```

Si deux classificateurs entrent en course sur le même titre, le perdant reçoit un 409 et re-lit la donnée ; pas d'écrasement silencieux. Le journal d'audit conserve qui (clé) a suggéré quoi — pratique pour annuler un modèle qui aurait eu une mauvaise journée.

11.6 Révoquer une clé

La révocation se fait en un clic dans le panneau d'administration. Une clé révoquée *ne peut pas* être restaurée ; l'historique d'audit des utilisations passées est préservé à côté de la ligne désormais désactivée. Si vous soupçonnez une fuite (commit dans un dépôt public, envoi par e-mail par erreur, présence dans un fichier de log), révoquez immédiatement et créez-en une nouvelle.

11.7 Ce que l'API n'expose *pas*

Par construction, l'API v1 est majoritairement en lecture. Elle ne permet pas :

- de créer ou supprimer des titres, volumes, contributeurs, emprunteurs, prêts ou séries ;
- de modifier un champ en-dehors de la liste blanche des quatre champs du PATCH ;
- de retourner des mots de passe, hachages, jetons de session ou autres credentials ;
- d'accepter des uploads multipart (pas de PUT de jaquette).

Ce sont des limites volontaires de la v1 — le cas d'usage qui justifie l'API est la classification, et cela tient dans la liste blanche restreinte. Une surface d'écriture plus large changerait la posture de sécurité (considérations CSRF, limitation de débit pour les mutations en masse) et est reportée tant qu'un besoin concret ne s'est pas manifesté.

Chapitre 12

Opérations & débogage

Ce chapitre documente l'emplacement des journaux sur disque, leur niveau de verbosité configurable, et les commandes courantes utiles quand un utilisateur signale un comportement étrange et que vous voulez voir ce que l'application a réellement fait.

12.1 Où sont les journaux

Depuis la **v1.7.0**, mybibli écrit ses journaux à deux endroits simultanément :

- **stdout** (inchangé) — accessible via `docker compose logs mybibli`. Idéal pour suivre le flux en direct pendant une session active.
- **Un fichier rotatif quotidien sur disque**, par défaut sous `/var/log/mybibli/` dans le conteneur, monté sur un volume nommé (`mybibli_logs`) côté hôte pour que les fichiers survivent à `docker compose pull && up -d`. Idéal pour le débogage *post-mortem*.

Les fichiers sont nommés `mybibli.log.AAAA-MM-JJ` (un par jour UTC). Une tâche en arrière-plan purge automatiquement les fichiers de plus de 30 jours.

Changer le chemin des journaux dans le conteneur

Le répertoire de journaux par défaut `/var/log/mybibli/` dans le conteneur est contrôlé par la variable d'environnement `MYIBLI_LOG_DIR`. Ne le changez que si vous remonte le volume ailleurs — le fichier `docker-compose.yml` fourni mappe le volume nommé `mybibli_logs` (ou un bind mount via `LOGS_HOST_PATH`) sur `/var/log/mybibli`, et changer l'un sans l'autre fait atterrir les journaux dans la couche inscriptible où ils sont effacés à chaque `docker compose up -d` suivant un pull.

Voir section 1.3 pour la configuration du bind mount (journaux visibles côté hôte pour DSM Log Center / journald shipping).

12.2 Lire les journaux depuis l'hôte

```
# Suivre en temps réel le fichier du jour
docker compose exec mybibli tail -f /var/log/mybibli/mybibli.log.$(date -u +%Y-%m-%d)

# Dernières 100 lignes du fichier du jour
docker compose exec mybibli tail -n 100 /var/log/mybibli/mybibli.log.$(date -u +%Y-%m-%d)

# Lister tous les jours disponibles
docker compose exec mybibli ls /var/log/mybibli/
```

```
# Rechercher un mot-clé sur tous les jours conservés
docker compose exec mybibli sh -c "grep ERROR /var/log/mybibli/mybibli.log.*"
```

Si vous avez monté le volume comme un bind mount (`LOGS_HOST_PATH` dans `.env`, voir section 1.3), les fichiers sont visibles directement depuis l'hôte sans passer par `docker compose exec`.

12.3 Niveaux de journalisation

Deux variables d'environnement contrôlent la verbosité :

- `MYBIBLI_LOG_LEVEL` (recommandée, v1.7.0+) — accepte soit un niveau simple (`trace`, `debug`, `info`, `warn`, `error`) soit une liste de directives `tracing-subscriber EnvFilter`, par exemple `mybibli=debug,sqlx::query=warn`.
- `RUST_LOG` — fallback historique, même format. Pris en compte si `MYBIBLI_LOG_LEVEL` est absent.

La valeur par défaut est `info` — sûre en production (`info`, `warn`, `error` ; pas de bruit SQL par requête, pas de flot de traces par requête HTTP).

Niveaux recommandés

info Défaut production. Capture les événements d'authentification, les récupérations de métadonnées, les tâches planifiées, les erreurs visibles utilisateur. Environ 10 à 100 lignes par jour pour une instance familiale.

`mybibli=debug` Investigation active d'une fonctionnalité mybibli. Ajoute le routage des requêtes, les cache hits/misses, les décisions du dispatcher de scan. Environ 10× le volume `info`.

`mybibli=trace` Investigation profonde ponctuelle. Inclut le tracing par appel dans le code mybibli. Très verbeux ; à utiliser brièvement puis revenir à `info`.

`mybibli=debug,sqlx::query=info` Investigation active ET voir chaque instruction SQL (utile quand une requête est suspecte).

Comment changer le niveau

Deux chemins, selon que vous voulez que le changement survive aux redémarrages du conteneur ou qu'il ne dure que le temps d'une investigation.

Depuis l'interface admin (v1.7.1+, recommandé)

Connectez-vous en Admin, allez sur `/admin?tab=system`, descendez jusqu'à la section **Journalisation**. Saisissez la nouvelle directive dans le champ **Niveau de journalisation** (tout ce que `tracing-subscriber::EnvFilter` accepte — niveau simple ou liste de directives). Cliquez **Enregistrer**. La prochaine ligne de log utilise le nouveau filtre — sans redémarrage du conteneur, sans `docker compose up -d`. La valeur est persistée en table `settings` et survit aux redémarrages.

Depuis `docker-compose.yml` / `.env`

Modifier le fichier, fixer `MYBIBLI_LOG_LEVEL`, puis :

```
docker compose up -d
```

Le conteneur redémarre et le nouveau niveau prend effet dès la prochaine ligne de journal. **À noter** : la valeur enregistrée en admin dans la table `settings` gagne au prochain boot. Donc si vous avez déjà enregistré un niveau depuis l’UI admin, c’est celui-là qui sera chargé — la variable d’environnement ne compte qu’à l’installation fraîche avant le premier enregistrement.

12.4 Lire le JSON structuré

Les journaux sont émis en JSON ligne par ligne. Chaque ligne porte :

- `timestamp` — ISO 8601 UTC.
- `level` — TRACE / DEBUG / INFO / WARN / ERROR.
- `target` — le module Rust qui a émis la ligne (par ex. `mybibli::routes::catalog`). Filtrer sur ce champ pour restreindre une recherche à un sous-système.
- `fields.message` — le résumé lisible.
- d’autres `fields.*` — clés de contexte comme `user_id`, `volume_id`, `isbn`, `title_id`.

Pour un affichage lisible :

```
docker compose exec mybibli tail -n 50 /var/log/mybibli/mybibli.log.$(date -u +%Y-%m-%d) \
| jq -r '[.timestamp, .level, .fields.message] | @tsv'
```

12.5 Joindre un extrait de log à un rapport de bug

Lors de l’ouverture d’une issue sur <https://github.com/guycorbaz/mybibli/issues> :

1. Notez l’heure murale où le bug s’est produit (par ex. “autour de 14h30 UTC aujourd’hui”).
2. Extrayez la fenêtre de 5 minutes concernée :

```
# Remplacer 14:25 / 14:35 par votre fenêtre.
docker compose exec mybibli sh -c \
"awk '/\"timestamp\": \"$(date -u +%Y-%m-%d)T14:2[5-9]/, /\"timestamp\": \"$(date -u +%Y-%m-%d)T14:3[0-5]/' \
/var/log/mybibli/mybibli.log.$(date -u +%Y-%m-%d)"
```

3. Copiez le résultat dans le corps de l’issue, balisé en ````json`.

Caviardez ce que vous estimez sensible — les lignes de journal peuvent contenir des noms d’emprunteurs, des ISBN de titres que vous possédez, ou des préfixes de jeton de session (mais jamais les jetons de session complets ni les hashes de mot de passe ; ils sont intentionnellement exclus de l’émission de journaux).

12.6 Rétention et rotation

- La rotation se fait **automatiquement** à minuit UTC, gérée en interne par `tracing-logger::rolling`.
- Les fichiers de plus de **30 jours** sont purgés une fois par jour par une tâche en arrière-plan. La fenêtre de rétention est codée en dur dans la v1.7.0 ; une version future l’exposera comme paramètre admin.

- Le fichier du jour n'est jamais supprimé (la coupure est stricte).

Pour conserver les journaux plus de 30 jours, copiez les fichiers rotatifs hors du volume vers du stockage long terme :

```
docker compose cp mybibli:/var/log/mybibli/. ./logs-archive/
```

Chapitre 13

Glossaire

Administrateur Compte utilisateur avec le rôle `admin`. Le seul rôle qui peut ouvrir le panneau `/admin` et éditer les utilisateurs, les données de référence, la corbeille ou les paramètres système.

Anonyme Visiteur sans cookie de session. Accès en lecture seule au catalogue public. Pas de catalogage, pas de prêt, pas d'édition.

Audit Le rituel « je parcours les étagères et je confirme que tout est à la place que la base indique » décrit en section [3.6](#).

BDGest Catalogue BD francophone utilisé comme source de métadonnées pour ce type de média.

BnF *Bibliothèque nationale de France*. Catalogue de la BNF, utilisé comme source principale de métadonnées pour les livres en français.

Catalogage L'acte d'enregistrer un nouveau titre et/ou volume dans la base. Typiquement piloté au code-barres : scanner, confirmer, coller l'étiquette, recommencer.

Code L Étiquette code-barres d'un lieu de stockage. Format : L suivi d'un numéro zéro-paddé.

Code V Étiquette code-barres d'un volume. Format : V suivi d'un numéro zéro-paddé.

Couverture La face visuelle avant d'un livre / disque / vinyle, téléchargée depuis un fournisseur de métadonnées et mise en cache sous `COVERS_DIR` sur disque.

CSP Content Security Policy. En-tête imposé par le navigateur qui restreint quels scripts et styles la page peut exécuter. mybibli applique une politique stricte qui interdit scripts et styles inline.

CSRF (jeton) Secret par session intégré dans chaque formulaire. mybibli rejette les requêtes POST / PUT / DELETE dont le jeton ne correspond pas — défense contre les forgeries de requête inter-sites.

Dewey (code) Le numéro de la Classification décimale de Dewey, utilisé pour la navigation par sujet du catalogue. Optionnel par titre.

EAN-13 Le standard de code-barres à 13 chiffres. Les ISBN sont encodés en EAN-13 depuis 2007 ; les CD, DVD et autres supports portent aussi des codes EAN-13.

- Emprunteur** Personne qui emprunte des volumes à la bibliothèque. Modélisé comme sa propre entité (nom, contact optionnel, historique de prêts).
- Exemplaire** Synonyme de *volume* (l'objet physique). Utilisé par certaines bibliothèques françaises ; mybibli préfère *volume*.
- État de volume** Ligne de données de référence décrivant l'état d'un volume : Neuf, Bon, Usé, Endommagé,
- Garde du dernier admin actif** Règle qui empêche de désactiver ou supprimer définitivement le dernier utilisateur de rôle `admin`. Voir section 2.6.
- Genre** Classification libre du sujet d'un titre (Fiction, Documentaire, Polar, ...). Définie dans la table de référence des genres.
- Hard-delete** Suppression d'une ligne pour de bon. mybibli ne hard-delete que depuis le panneau `/admin > Trash` et la tâche d'auto-purge en arrière-plan ; tout le reste est un soft-delete.
- HTMX** Bibliothèque JavaScript qu'utilise mybibli pour échanger des fragments de page sans rechargement complet. Les exploitants n'ont pas besoin de comprendre HTMX pour utiliser mybibli.
- ISBN** International Standard Book Number. Identifiant imprimé au dos des livres, encodé en code-barres EAN-13.
- Lieu** L'endroit physique où vit un volume. Modélisé comme un arbre (pièce → étagère → rangée), identifié par un code L.
- MariaDB** Le moteur de base où mybibli stocke ses données. Fork de MySQL.
- Multi-position (volume)** Un volume physique unique qui contient plusieurs positions dans une série — typiquement un album BD omnibus.
- MusicBrainz** Catalogue de métadonnées musicales maintenu par des bénévoles, utilisé comme source principale pour les médias audio.
- OMDb** Open Movie Database. Source de métadonnées pour films et séries TV.
- Open Library** Catalogue ouvert de livres porté par des bénévoles, utilisé comme source de métadonnées.
- Optimistic locking** Technique qu'utilise mybibli pour détecter les éditions concurrentes. Deux utilisateurs qui éditent le même titre obtiennent une erreur claire à la seconde sauvegarde au lieu de s'écraser silencieusement.
- Prêt** Enregistrement qu'un volume précis est actuellement sorti chez un emprunteur précis. Note la date de prêt, la date de retour prévue, la date de retour, et le lieu pré-prêt du volume.
- Réactivation** Restauration d'une ligne soft-deleted. Pour les utilisateurs, efface `deleted_at` et leur permet de se reconnecter. Pour les données de référence, le terme couvre aussi le fait de re-crée une ligne avec le même nom qu'une soft-deleted, ce qui défait la suppression de manière transparente.
- Série** Suite ordonnée et nommée de titres liés (par exemple une trilogie ou une série BD en cours). La position de chaque titre dans la série est un petit entier ou une plage (pour les omnibus).

Setup wizard Le flux en quatre étapes à `/setup` qu’une instance vide présente jusqu’à ce qu’un compte admin existe.

Soft-delete Positionnement de la colonne `deleted_at` d’une ligne à « maintenant » au lieu de supprimer physiquement la ligne. Les lignes soft-deleted sont invisibles aux requêtes normales mais apparaissent dans `/admin > Trash` pendant 30 jours, après quoi l’auto-purge les hard-delete.

Titre L’entité bibliographique (un livre, un film, un album). Un titre peut avoir plusieurs volumes — par exemple, deux exemplaires du même roman sur des étagères différentes.

TMDb The Movie Database. Source de métadonnées communautaire pour films et séries TV.

Type de média La nature de l’objet que représente un titre : Livre, BD / comic, Audio, Film / série. Pilote la sélection des fournisseurs et l’affichage des champs.

Type de nœud Libellé appliqué aux lieux de stockage pour indiquer quel type de contenant ils sont (pièce, étagère, rangée, boîte). Pur libellé — aucune contrainte sémantique.

Volume L’objet physique sur l’étagère — un exemplaire d’un titre. Un titre peut avoir plusieurs volumes ; chacun porte son propre code V, son lieu et son état.

Index

- administrateur, 26, 57
- anonyme, 26, 57
- arbre des lieux, 10
- assistant de mise en service, 6, 9
- audit, 18, 57

- BD / comic, 20
- BDGest, 22, 57
- bibliothécaire, 26
- BnF, 22, 57

- champ de scan, 15
- code L, 11, 57
- code V, 11, 57
- Code-128, 12
- cookie de session, 27
- couvertures, 57
 - persistance, 6
 - répertoire, 4
- CSRF, 57

- Dewey, 42, 57
- dépannage
 - DATABASE_URL, 44

- EAN-13, 12
- emprunteur, 17, 58

- film, 20
- fournisseurs de métadonnées, 3, 22

- garde du dernier admin, 58
- genres, 13, 58
- Google Books, 22

- i18n, 13
- installation, 3

- journaux
 - répertoire, 4

- lieu, 15, 58
- lieux de stockage, 10

- mise à jour 1.1.0, 34
- mode HID clavier, 12
- multi-position, 20, 58
- MusicBrainz, 21, 22, 58
- MYBIBLI_SEED_DEV_USERS, 35

- OMDb, 21, 22, 58
- omnibus, 20
- Open Library, 22, 58

- panneau admin, 10
- prêt, 58

- rangement, 18
- rôles, 14, 26
- rôles de contributeur, 13

- sauvegarde
 - .env, 29
 - base, 29
 - couvertures, 29
- setup wizard, 59
- soft-delete, 59
- sécurité
 - seuil 1.1.0, 3
- série, 20, 58

- timeout d'inactivité, 27
- TMDb, 21, 22, 59
- type de nœud, 11, 13, 59
- types de médias, 20, 59

- utilisateur SYSTEM, 45

- variables d'env
 - .env.example, 7
 - clés fournisseurs, 24
- volume, 10, 11, 59

- wish list, 18
- /wishlist, 18

- état de volume, 13, 15, 58